

A HUMAN-LIKE AGENT FOR SCREEN-AWARE TASK AUTOMATION

25-26J-477

BSc (Hons) Degree in Information Technology
Specializing in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

A HUMAN-LIKE AGENT FOR SCREEN-AWARE TASK AUTOMATION

25-26J-477

Dissertation submitted in partial fulfillment of the requirements for the Bachelor of
Science (Hons) in Information Technology Specializing in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

DECLARATION

We declare that this is our own work and this dissertation does not incorporate, without acknowledgement, any material previously submitted for a Degree or Diploma in any other University or institute of higher learning. To the best of our knowledge and belief, it does not contain any material previously published or written by another person except where due acknowledgement is made in the text.

We also hereby grant to the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this dissertation, in whole or in part, in print, electronic, or other media. We retain the right to use this content, in whole or in part, in future works (such as articles or books).

Signature

Date



25/04/2026

.....

.....

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Signature of the supervisor

Date



25/04/2026

.....

.....

ABSTRACT

This research presents the design and implementation of *A Human-Like Agent for Screen-Aware Task Automation*, an Agentic AI-based system developed to enable autonomous execution of user interface tasks using natural language instructions. The primary objective of the system is to achieve intelligent, adaptive, and reliable GUI automation through a closed-loop interaction cycle consisting of perception, planning, action, and evaluation.

The proposed system integrates multiple core components, including visual perception, LLM-based prompt engineering and action planning, HID-based actuation, and an Agentic AI control mechanism. Together, these components enable the system to interpret user intent, understand on-screen environments through external visual input, generate structured action plans, and execute them on the target system without requiring internal APIs or system-level access.

At the core of the system is a feedback-driven agentic loop that continuously monitors execution outcomes using real-time visual feedback. This allows the system to validate actions, detect failures, and dynamically adapt its behavior through re-planning and retry mechanisms. Such adaptability enhances robustness in dynamic and unstructured GUI environments where traditional automation approaches often fail.

Unlike conventional automation frameworks that rely on static scripts or predefined workflows, the proposed system supports flexible, context-aware execution in black-box environments, including legacy systems. By leveraging vision-based grounding and intelligent decision-making, the system demonstrates human-like interaction capabilities across diverse interface scenarios.

The integration of perception, reasoning, and actuation within a unified architecture provides a scalable and extensible foundation for autonomous UI automation. Experimental results indicate that the system improves reliability, adaptability, and execution effectiveness in real-world testing scenarios, making it a promising approach for next-generation intelligent automation systems.

Keywords – Agentic AI, GUI Automation, Visual Perception, Human Interface Device (HID), Black-Box Testing, Autonomous Systems, Prompt Engineering

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who supported us throughout the successful completion of our final year research project.

First, we extend our heartfelt appreciation to our research supervisor, Mr. Vishan Jayasinghearachchi, for his continuous guidance, encouragement, and insightful feedback throughout this study. His support played a crucial role in shaping the direction and quality of this research.

We are also deeply thankful to our co-supervisor, Ms. Kaushalya Rajapaksa, for her valuable suggestions, constructive feedback, and support during the planning and development stages of this project.

Furthermore, we would like to acknowledge each member of our research group for their collaboration, dedication, and shared commitment in achieving the project objectives.

Finally, we express our deepest gratitude to our parents and friends for their constant encouragement, understanding, and unwavering support throughout this journey.

TABLE OF CONTENTS

DECLARATION	<i>i</i>
ABSTRACT	<i>ii</i>
ACKNOWLEDGEMENT.....	<i>iii</i>
LIST OF FIGURES	<i>vi</i>
LIST OF TABLES	<i>vii</i>
LIST OF ABBREVIATIONS	<i>viii</i>
1. INTRODUCTION.....	1
1.1 General Introduction	1
1.2 Background Literature	2
1.3 Limitation in the Current Approach	4
1.4 Research Gap	5
2. RESEARCH PROBLEM.....	9
2.1 Problem Context	9
2.2 Problem Statement	10
2.3 Core Research Problem.....	12
3. RESEARCH OBJECTIVES	13
3.1 Main Objective.....	13
3.2 Specific Objective	14
4. METHODOLOGY	15
4.1 System Architecture	16
4.2 Specific Component Description	20
4.2.1 Purpose	22
4.2.2 Workflow Overview	24
4.2.3 Input Processing.....	26
4.2.4 Context Structuring and Decision Preparation	27
4.2.5 Action Planning and Decision Generation	28
4.2.6 Perception-Driven Validation.....	28
4.2.7 Execution and Error Handling	29
4.2.8 Exploratory Branching.....	30
4.2.9 Optimization and Evaluation.....	30
4.3 Agentic Decision-Making and Action Planning Framework.....	31
4.3.1 Instruction Processing and Context Initialization	32
4.3.2 Context-Aware Reasoning Strategy	32
4.3.3 Action Decomposition and Planning.....	32
4.3.4 Structured Action Representation	33
4.3.5 Action Validation and Safety Checks.....	33

4.3.6	Feedback-Driven Adaptation	34
4.3.7	Continuous Optimization of Decision Strategy	34
4.4	Design System Diagrams.....	35
4.4.1	DFD Diagram	36
5.	IMPLEMENTATION AND TESTING	36
5.1	Implementation	36
5.1.1	System Component Overview.....	37
5.1.2	Instruction-to-Action Coordination Pipeline.....	39
5.1.3	Visual Context Construction and State Validation	40
5.1.4	Execution and HID Control Layer	41
5.1.5	Feedback Loop and Evaluation Mechanism.....	41
5.2	Testing	43
5.2.1	Test Plan and Strategy	44
5.2.2	Test Case Design	45
6.	RESULTS AND DISCUSSIONS.....	49
6.1	Results	49
6.1.1	Overall Execution Outcomes.....	51
6.1.2	Quantitative Performance Summary	56
6.1.3	Accuracy and Output Structure Quality	57
6.1.4	Robustness Under Edge Condition	60
6.1.5	Comparative Behavior Across Scenario Types	61
6.1.6	Practical Result Interpretation	62
6.2	Research Finding	63
6.3	Discussion	66
7.	CONCLUSIONS.....	71
8.	REFERENCES.....	74
9.	APPENDICES.....	76

LIST OF FIGURES

Figure 4.1: Overview of system architecture diagram	16
Figure 4.2: High-level layered architecture diagram	19
Figure 4.3: Component diagram	24
Figure 4.4: Processing workflow	26
Figure 4.5: DFD diagram	36
Figure 5.1: Endpoint flow diagram	38
Figure 5.2: Agentic instruction-to-action coordination flow.....	40
Figure 5.3: HID execution and retry mechanism flow	41
Figure 6.1: Agentic system execution output (run JSON file).....	52

LIST OF TABLES

Table 1.1: Limitations and Proposed Directions.....	7
Table 4.1: Technologies, techniques, architecture and algorithms used.....	18
Table 5.1: End-to-End Login Task Execution Using Agentic Loop.....	45
Table 5.2: Visual Perception and UI Element Detection.....	46
Table 5.3: LLM-Based Task Planning and Action Generation.....	47
Table 5.4: HID Execution and Retry Mechanism.....	48
Table 5.5: Feedback-Driven Evaluation and Recovery Mechanism.....	48

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
LLM	Large Language Model
API	Application Programming Interface
UI	User Interface
GUI	Graphical User Interface
OCR	Optical Character Recognition
HID	Human Interface Device
RPA	Robotic Process Automation
VLM	Vision-Language Model
CV	Computer Vision
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
REST	Representational State Transfer
DFD	Data Flow Diagram
QA	Quality Assurance
QA	Quality Assurance Testing
ESP32	Low-cost microcontroller with Wi-Fi and Bluetooth capabilities
OCR	Optical Character Recognition
VLM	Vision-Language Model
Agentic AI	An AI system capable of autonomous decision-making through perception, planning, action, and feedback loops
Prompt Engineering	The practice of designing and optimizing inputs to guide LLM outputs effectively
Perceive–Plan–Act–Evaluate Loop	An iterative agent framework for continuous decision-making and execution in dynamic environments

1. INTRODUCTION

1.1 General Introduction

A Human-Like Agent for Screen-Aware Task Automation is an agentic artificial intelligence-based system designed to perform human-like interactions with graphical user interfaces (GUIs) using natural language instructions. The system enables autonomous task execution in environments where traditional automation techniques are ineffective, such as legacy systems, secure platforms, or applications without accessible APIs. Instead of relying on internal system integration, the proposed solution operates through external visual observation and hardware-level input emulation, making it suitable for black-box testing scenarios [1], [2].

The system follows a closed-loop workflow consisting of perception, planning, action, and evaluation. In the perception stage, a visual perception module captures screen data using an external camera and processes it using computer vision and optical character recognition (OCR) techniques to extract structured information about UI elements [3]. This structured representation serves as the environmental context for subsequent decision-making.

In the planning stage, a dedicated LLM and prompt engineering module interprets user instructions and combines them with the extracted visual context to generate structured, machine-readable action plans. These plans are decomposed into step-by-step UI interactions, ensuring that each action is grounded in the current screen state and aligned with the intended task objectives [4].

The action stage is handled by a Human Interface Device (HID) actuation module, which translates high-level action steps into low-level input commands such as mouse movements, clicks, and keyboard inputs. These commands are executed on the target system using hardware-based input emulation, enabling real-time interaction without requiring direct software integration.

At the core of the system is the Agentic AI module, which orchestrates the entire perceive–plan–act–evaluate loop. It coordinates communication between all components, maintains task context, monitors execution progress, and evaluates outcomes based on visual feedback. By continuously analyzing the updated UI state after each action, the system dynamically adapts its behavior, handles errors, and triggers re-planning or retry mechanisms when necessary.

The overall architecture follows a microservices-based design, where the visual perception, LLM reasoning, and HID execution components operate as independent services. The Agentic AI module serves as the central orchestrator, ensuring synchronized operation and efficient data flow between modules. This modular approach enhances scalability, flexibility, and maintainability, aligning with modern software engineering practices [5].

By integrating perception, intelligent reasoning, and physical actuation within a unified agentic framework, the system enables adaptive and context-aware automation. Unlike traditional rule-based or script-driven approaches, ScreenPilot supports flexible execution in dynamic and unpredictable GUI environments. The implementation demonstrates the potential of agentic systems to advance autonomous UI interaction and contributes to ongoing research in intelligent automation and human–computer interaction [6].

1.2 Background Literature

Graphical User Interface (GUI) automation has been widely studied as a means of improving software testing efficiency and reducing repetitive human interaction. Traditional automation approaches, such as WebDriver-based tools and rule-based Robotic Process Automation (RPA), depend heavily on internal system access, including Document Object Model (DOM) structures, application programming interfaces (APIs), or accessibility trees [7], [8]. While these techniques are highly effective in controlled environments, their applicability is limited in black-box scenarios where internal system details are inaccessible or restricted [9].

To address these limitations, vision-based automation methods have been introduced, focusing on interpreting user interfaces through visual data. Techniques based on computer vision and Optical Character Recognition (OCR), such as those implemented in Tesseract OCR, enable extraction of textual and structural information from screen images or camera feeds [10]. Additionally, datasets like Rico have contributed to modeling UI layouts and understanding interface patterns [11]. Although these approaches improve perception capabilities, they primarily focus on detecting UI elements and lack integrated mechanisms for decision-making, task planning, and coordinated execution.

Recent developments in Large Language Models (LLMs) have significantly advanced the field of intelligent automation. Models such as Gemini demonstrate strong capabilities in understanding natural language instructions and generating structured action sequences through prompt engineering and reasoning techniques [12], [13]. Agentic paradigms, including frameworks like ReAct, further extend these capabilities by enabling iterative reasoning and action cycles. However, many LLM-based solutions operate in simulated environments or rely on API-level interactions, which limits their effectiveness in real-world scenarios that require direct interaction with graphical interfaces and continuous environmental feedback [14], [15].

At the execution level, hardware-based interaction approaches using Human Interface Device (HID) emulation provide a practical method for interacting with target systems. Microcontrollers such as ESP32 can simulate keyboard and mouse inputs, allowing automation without modifying the underlying software [16], [17]. While these methods enable non-intrusive interaction in restricted environments, they lack built-in intelligence and require higher-level coordination to determine when and how actions should be executed.

Overall, existing research demonstrates significant progress in individual domains of perception, reasoning, and execution. However, these components are often developed in isolation, resulting in fragmented solutions that do not support full end-to-end automation. There remains a clear gap in integrating visual perception, language-based reasoning, and physical execution within a unified, feedback-driven system.

This gap highlights the need for an agentic framework that combines perception, planning, action, and evaluation into a continuous closed-loop process. Such a system would enable adaptive decision-making, real-time feedback utilization, and robust task execution in dynamic and unpredictable GUI environments. The proposed approach addresses this gap by integrating all core components into a cohesive architecture, enabling practical and scalable automation in black-box scenarios.

1.3 Limitation in the Current Approach

Although the proposed system demonstrates the feasibility of intelligent screen-aware automation, several limitations remain across the integrated components, particularly in perception accuracy, reasoning reliability, execution stability, and overall system coordination.

First, the system is highly dependent on the accuracy of the visual perception component. The vision module relies on camera-based input and OCR techniques to extract user interface elements and their spatial properties. Environmental factors such as lighting variations, camera positioning, motion blur, and screen reflections can introduce noise and detection errors. These inaccuracies directly affect downstream components, as incorrect or incomplete visual data may lead to improper action planning and execution.

Second, the instruction interpretation and planning process, driven by large language models and prompt engineering, introduces inherent uncertainty. While structured prompts are used to enforce output formats, the probabilistic nature of LLMs can still result in ambiguous, incomplete, or inconsistent action plans. This limitation affects the reliability of generated steps, particularly in complex or poorly defined tasks, and places additional responsibility on validation and orchestration mechanisms to ensure correctness.

Third, the system exhibits limitations in real-time responsiveness due to its multi-stage processing pipeline. The sequential flow of perception, context construction, reasoning, action generation, and HID execution introduces cumulative latency. This delay can impact

the system’s ability to respond promptly to dynamic interface changes, reducing effectiveness in time-sensitive or rapidly evolving environments.

Fourth, the execution layer, which depends on hardware-based Human Interface Device (HID) emulation, introduces practical challenges related to communication and synchronization. Issues such as connection instability, input lag, or focus misalignment on the target system can disrupt execution. Additionally, the lack of guaranteed feedback from the host system makes it difficult to ensure that actions have been performed exactly as intended.

Fifth, while the Agentic AI Logic implements a feedback-driven loop, the current evaluation mechanism remains relatively shallow. The system primarily relies on surface-level comparisons of screen states and execution status, without deeper semantic verification of task success. As a result, certain failures—such as partial input completion or incorrect field interactions—may not be fully detected or corrected.

Finally, the system has been evaluated primarily in controlled environments with limited variability in interface design and workflow complexity. This restricts the generalizability of the results. Real-world applications may involve diverse layouts, varying resolutions, dynamic content, and multi-step dependencies, which the current implementation may not handle robustly without further enhancements.

Overall, these limitations highlight the challenges of integrating perception, reasoning, and execution into a unified agentic framework. Addressing these issues will require improvements in perception robustness, prompt design, state management, execution verification, and system-level optimization to support more reliable and scalable automation in real-world environments.

1.4 Research Gap

Despite significant advancements in automation technologies, computer vision, and Large Language Models (LLMs), a critical gap remains in the development of fully integrated,

agent-driven frameworks for real-world graphical user interface (GUI) automation. Existing approaches typically address perception, reasoning, and execution as isolated components, with limited emphasis on how these elements can be cohesively combined into a unified, feedback-driven system.

One major limitation is the lack of a centralized orchestration mechanism capable of coordinating multiple system components in real time. While vision-based modules can extract structured interface data and LLM-based systems can generate high-level action plans, there is insufficient research on how these outputs are synchronized, validated, and executed within a dynamic environment. Without a cohesive control layer, system behavior tends to remain fragmented, leading to inconsistencies and reduced reliability during task execution.

Another key gap lies in the absence of robust feedback-driven control loops. Many existing automation systems follow a linear execution model, where actions are performed sequentially without continuous validation of their outcomes. In real-world GUI environments, interface states can change dynamically due to asynchronous updates, rendering delays, or layout variations. The lack of iterative evaluation and re-planning mechanisms limits the system’s ability to detect failures, recover from errors, and adapt execution strategies accordingly.

Furthermore, there is a disconnect between high-level reasoning and low-level execution. LLMs are capable of generating abstract and structured action sequences; however, these plans are not always aligned with real-time visual context or physical execution constraints. Similarly, hardware-based interaction mechanisms such as HID emulation enable low-level input execution but lack awareness of task semantics or environmental state. This separation highlights the need for a unified framework that can bridge semantic planning with grounded execution.

In addition, existing systems provide limited support for adaptive retry mechanisms and exploratory behavior. When execution failures occur, most frameworks either terminate the process or repeat identical actions without modification. There is a lack of intelligent strategies for adjusting actions, exploring alternatives, or leveraging feedback from

previous attempts. This limits the system’s ability to behave in a human-like, resilient manner when interacting with unpredictable interfaces.

Another important gap is related to real-world applicability. Many existing solutions are evaluated in controlled or simulated environments with clean inputs and predictable behaviors. In contrast, real-world scenarios involve noisy visual data, incomplete interface detection, inconsistent layouts, and limited feedback signals. Current approaches do not sufficiently address how to maintain robustness under such conditions.

Moreover, while individual components such as vision-based perception, prompt-driven reasoning, and HID-based execution have been studied independently, there is limited work on integrating these components into a cohesive architecture. In particular, the coordination between visual context construction, instruction interpretation, decision-making, and physical execution remains an underexplored area.

Therefore, a clear research gap exists in the design and implementation of a unified agentic framework that integrates visual perception, language-based reasoning, decision orchestration, and hardware-level execution within a continuous closed-loop system. Such a framework must support real-time feedback integration, context-aware planning, adaptive execution strategies, and robust inter-component coordination to enable reliable automation in complex, dynamic, and black-box GUI environments.

Table 1. 1: Limitations and Proposed Directions

Aspect	Current Limitation	Impact	Proposed Direction
System Coordination	Lack of unified integration across components	Fragmented execution flow	Introduce centralized agentic orchestration
Visual Perception	Sensitivity to noise and environmental factors	Inaccurate UI detection	Improve robustness of vision and

			OCR processing
Instruction Processing	Ambiguity in LLM-generated outputs	Inconsistent action plans	Enhance prompt engineering and validation
Execution Strategy	Static and linear workflows	Poor adaptability	Enable dynamic, context-aware decision-making
Feedback Handling	Limited evaluation mechanisms	Failure detection issues	Implement closed-loop evaluation and verification
Error Recovery	No intelligent retry mechanisms	Reduced robustness	Introduce adaptive retry and exploration strategies
Real-world Applicability	Evaluation in controlled environments	Limited generalization	Design for noisy, black-box scenarios

The proposed system addresses these gaps by introducing an integrated architecture that combines visual perception, LLM-based instruction processing, agentic decision-making,

and HID-based execution within a unified closed-loop framework. By enabling continuous feedback, dynamic planning, and adaptive execution, the system improves robustness, flexibility, and reliability for real-world GUI automation in access-restricted environments.

2. RESEARCH PROBLEM

2.1 Problem Context

Agentic artificial intelligence has emerged as a promising paradigm for enabling autonomous systems to perform complex tasks through continuous interaction with their environment. In the context of GUI automation, this involves understanding visual interface states, generating appropriate actions, executing them through external input mechanisms, and evaluating outcomes in a coordinated feedback loop. Unlike traditional rule-based automation, agentic systems aim to exhibit adaptive, human-like behavior by responding dynamically to changing conditions and incomplete or uncertain information.

In modern graphical user interface (GUI) environments, automation is particularly challenging due to the lack of stable internal access in black-box systems. Many applications do not expose APIs, DOM structures, or accessibility trees, making conventional automation approaches ineffective. These environments are inherently dynamic, with frequently changing layouts, varying screen content, and unpredictable response behaviors. As a result, reliable automation requires more than isolated perception or execution—it demands continuous reasoning, context maintenance, and adaptive decision-making throughout the entire interaction cycle.

Although individual technologies such as vision-based perception, LLM-driven instruction understanding, prompt-based planning, and Human Interface Device (HID) execution have matured significantly, they are typically developed as independent modules. The absence of tight integration between these components creates a major limitation in real-world

deployment. Without a unified coordination mechanism, each stage operates in isolation, leading to inconsistencies between perceived state, planned actions, and executed outcomes.

Furthermore, most existing approaches lack a structured agentic control layer capable of managing the full lifecycle of task execution. There is insufficient support for real-time feedback integration, iterative correction, and dynamic re-planning based on execution results. This leads to fragile automation pipelines that struggle to recover from errors, adapt to interface changes, or handle incomplete visual information.

In addition, the disconnect between high-level reasoning and low-level execution further amplifies the problem. While LLM-based systems can generate logically sound action sequences, these plans are not always grounded in real-time visual context or constrained by execution feasibility. Similarly, hardware-based HID execution operates at a mechanical level without awareness of task intent or environmental state, resulting in a gap between decision-making and physical interaction.

Therefore, the core research problem lies in the absence of a unified, feedback-driven agentic framework that can seamlessly integrate visual perception, language-based reasoning, decision orchestration, and hardware-level execution into a single closed-loop system. Such a system is essential for enabling reliable, adaptive, and context-aware GUI automation in complex, dynamic, and black-box environments.

2.2 Problem Statement

Despite significant advancements in computer vision, large language models (LLMs), and hardware-based execution mechanisms, existing GUI automation systems still lack a unified Agentic AI framework capable of coordinating perception, reasoning, and execution within a continuous, feedback-driven control loop.

A fundamental limitation is the absence of a centralized orchestration mechanism to manage interactions between visual perception, language-based planning, and low-level

execution. In most current systems, these components are implemented as independent modules, leading to fragmented workflows where decisions generated by reasoning components are not consistently aligned with real-time visual state or execution constraints. This disconnection reduces reliability and limits the system’s ability to function adaptively in dynamic environments.

Another critical issue is insufficient context management across multi-step workflows. GUI automation tasks typically require sequential interactions where each step depends on the outcome of previous actions. However, without a persistent and structured mechanism for maintaining execution state, systems often lose contextual continuity. This results in redundant actions, incomplete task execution, or incorrect decision paths, particularly in long or complex workflows.

Furthermore, existing approaches lack effective real-time evaluation and feedback integration. Most automation pipelines execute actions without continuously validating whether the intended effect has been achieved. As a result, failures such as incorrect field selection, partial input execution, or UI state mismatches are often not detected early, limiting the system’s ability to self-correct during execution.

In addition, adaptive decision-making and intelligent retry mechanisms are largely underdeveloped. When an action fails, many systems either terminate execution or repeat the same operation without modification. This rigid behavior is unsuitable for real-world GUI environments, where interface states may change dynamically and require alternative strategies or exploratory actions to complete the task successfully.

Therefore, the core problem addressed in this research is the absence of a unified Agentic AI Logic framework that can integrate visual perception, LLM-based reasoning, decision orchestration, and HID-based execution into a single, continuous, and feedback-driven system. The proposed solution aims to enable context-aware decision-making, dynamic adaptation, and robust error recovery, thereby improving the reliability and effectiveness of GUI automation in complex, black-box environments.

2.3 Core Research Problem

Therefore, the core research problem addressed in this study is the lack of a unified and robust Agentic AI framework capable of orchestrating visual perception, language-based reasoning, action planning, and hardware-level execution within a continuous, feedback-driven control loop for GUI automation.

Although existing systems are capable of interpreting user instructions and generating structured action sequences, they do not provide a centralized mechanism to coordinate the complete execution lifecycle. As a result, there is limited support for maintaining contextual awareness across multi-step workflows, ensuring consistency between perceived interface states and generated actions, and validating whether executed commands have achieved their intended outcomes.

A further challenge lies in the absence of real-time adaptive decision-making. Current approaches typically execute pre-generated plans without continuously adjusting to changes in the environment. This becomes problematic in dynamic GUI settings, where interface elements may shift, load asynchronously, or change state unexpectedly. Without an intelligent coordination layer, systems are unable to reliably recover from partial failures or unexpected execution results.

In addition, existing solutions do not adequately integrate feedback from execution back into the decision-making process. This limits the ability to perform iterative refinement, error correction, and re-planning based on actual system behavior. Consequently, automation pipelines often remain rigid and are unable to function effectively in uncertain or black-box environments.

To address these limitations, a control framework is required that can seamlessly integrate perception outputs, LLM-generated plans, and HID-based execution while maintaining a continuous evaluation loop. This framework must ensure structured decision-making, persistent task context management, and adaptive response strategies in order to maintain execution reliability across complex workflows.

Addressing this core problem is essential for enabling scalable, resilient, and intelligent GUI automation systems capable of operating under real-world uncertainty. The development of such an agentic coordination mechanism forms a critical step toward achieving autonomous, human-like interaction in modern interface-driven applications.

3. RESEARCH OBJECTIVES

3.1 Main Objective

The primary objective of this study is to design and implement a robust Agentic AI-based automation framework that enables autonomous, context-aware GUI interaction through a continuous perceive–plan–act–evaluate loop. The system integrates visual perception, language-based reasoning, decision orchestration, and hardware-level execution to achieve reliable and adaptive task completion in dynamic and unstructured environments.

The proposed framework aims to bridge the gap between high-level user intentions and low-level interface execution by introducing an intelligent coordination mechanism. This mechanism is responsible for interpreting real-time visual context, generating structured action plans using LLM-based reasoning, validating execution feasibility, and translating decisions into HID-based input commands for physical interaction with the system.

A key focus of the system is maintaining continuous task awareness across multiple steps. It ensures that perception outputs are accurately interpreted, planning decisions are contextually grounded, and execution results are continuously evaluated to support adaptive decision-making. This allows the system to handle both deterministic workflows and ambiguous or exploratory tasks with improved reliability.

Furthermore, the framework is designed to operate effectively in black-box environments where internal system access such as APIs, DOM structures, or accessibility layers is unavailable. By relying solely on external visual feedback and input emulation, the system

ensures broad applicability across legacy systems, secure applications, and heterogeneous platforms.

Ultimately, the objective is to develop a scalable and intelligent agentic automation system capable of continuously monitoring, evaluating, and refining its behavior. This enables more human-like interaction, improved robustness under uncertainty, and practical deployment in real-world GUI automation scenarios.

3.2 Specific Objective

The specific objectives of this study are as follows:

- To design and implement a closed-loop agentic framework based on the Perceive–Plan–Act–Evaluate cycle, enabling continuous interaction between visual perception, language-based reasoning, decision orchestration, and hardware-level execution components.
- To develop a vision-based perception module capable of extracting structured UI elements (such as labels, coordinates, and element types) from screen input, enabling accurate understanding of the current interface state.
- To design and implement an LLM-based instruction processing and planning module that converts natural language user tasks into structured, executable action sequences through prompt engineering and contextual reasoning.
- To develop a centralized Agentic AI Logic module that coordinates perception outputs, planning decisions, and execution feedback, ensuring consistent and context-aware task execution across all system components.
- To implement context management mechanisms that preserve task state across multi-step workflows, enabling sequential consistency and continuity during complex GUI interactions.
- To design a feedback-driven evaluation system that validates executed actions using real-time visual updates, ensuring alignment between intended actions and actual interface outcomes.

- To integrate a hardware-based Human Interface Device (HID) execution module capable of translating structured action plans into low-level mouse and keyboard commands for real-world system interaction.
- To develop adaptive retry and decision refinement strategies that allow the system to recover from execution failures, handle unexpected UI changes, and dynamically adjust action plans based on feedback.
- To ensure synchronized end-to-end operation across all components, enabling seamless coordination between perception, reasoning, decision-making, and execution layers.
- To enable robust operation in black-box environments by relying exclusively on external visual feedback and coordinated agentic decision-making, without requiring internal system access such as APIs or DOM structures.

4. METHODOLOGY

The methodology of the proposed Agentic AI-based automation system is designed to enable intelligent, feedback-driven GUI interaction through a unified closed-loop control mechanism. Unlike traditional automation pipelines that operate in a linear or rule-based manner, this approach emphasizes continuous perception, reasoning, execution, and evaluation under the coordination of a centralized agentic controller.

The system follows a structured Perceive–Plan–Act–Evaluate workflow, where each stage is tightly integrated to ensure context-aware and adaptive task execution. The Agentic AI Logic module acts as the core orchestration unit, responsible for maintaining task context, coordinating subsystem interactions, and dynamically adjusting decisions based on real-time feedback from the environment.

4.1 System Architecture

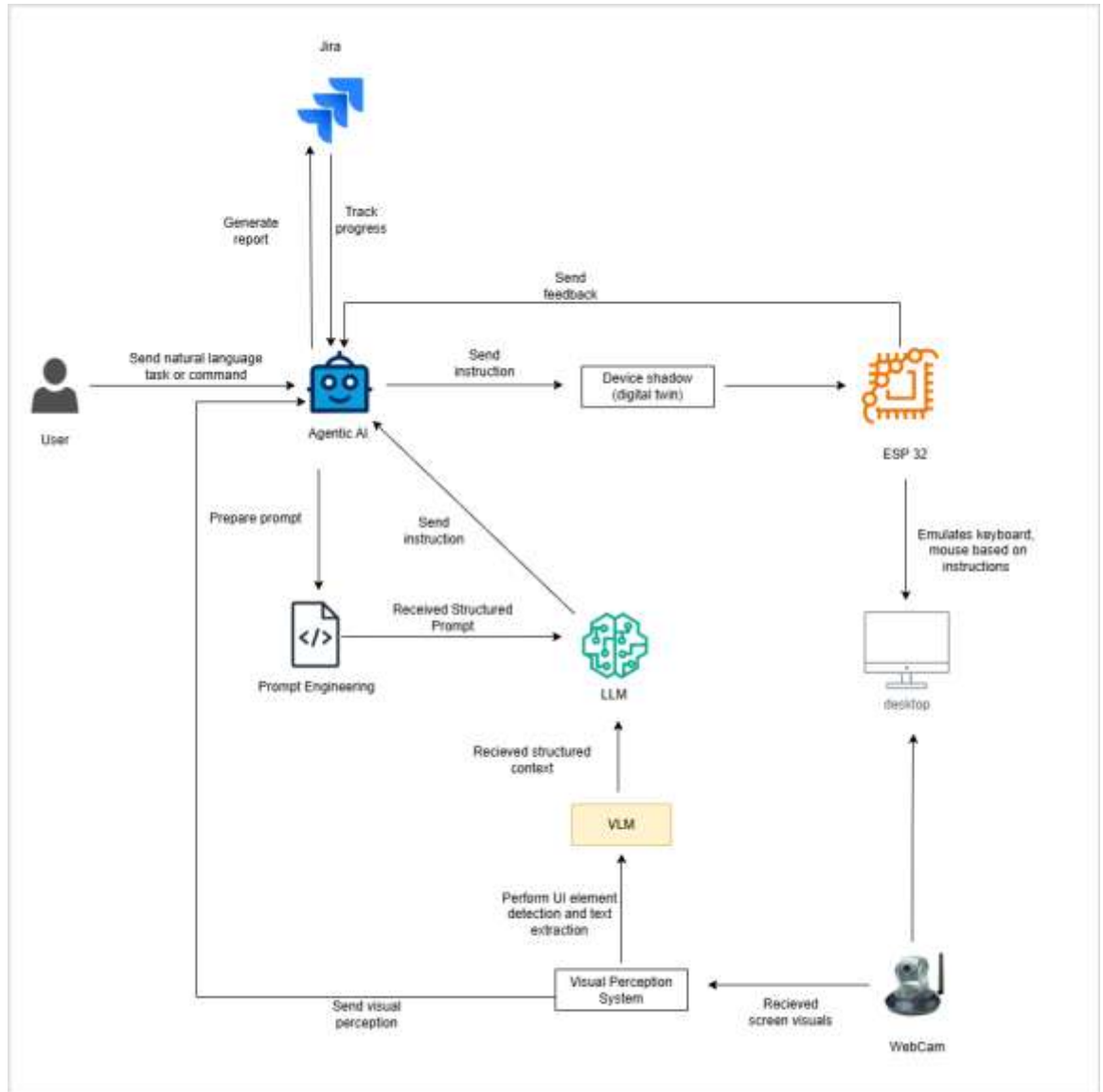


Figure 4. 1: Overview of system architecture diagram

Figure 4.1 illustrates the overall architecture of the agentic automation system, which is designed to operate in black-box GUI environments using external perception and hardware-based execution. The system is composed of four tightly integrated components: the Vision Module, LLM & Prompt Engineering Module, Agentic AI Logic Module, and HID Execution Module.

At the center of the architecture is the **Agentic AI Logic module**, which functions as the central decision-making and orchestration layer. It receives natural language instructions from the user interface and initiates the execution cycle by maintaining task context and coordinating all other system components.

The **visual perception module** captures real-time screen data using an external camera and processes it using computer vision techniques and OCR. It extracts structured information such as UI element types, labels, coordinates, and states. This structured visual representation is then forwarded to the Agentic AI Logic module, enabling environment-aware reasoning.

The **LLM and prompt engineering module** is responsible for converting user instructions and visual context into structured action plans. Using carefully designed prompts, the model generates step-by-step instructions in a machine-readable format. These outputs are then validated and interpreted by the Agentic AI Logic module before execution.

Once an action is approved, the **HID execution module** translates structured commands into low-level input events such as mouse movements, clicks, and keyboard inputs using an ESP32-based Human Interface Device. This enables platform-independent interaction with the target system without requiring internal access or APIs.

A key feature of this architecture is the **feedback-driven evaluation loop**. After each action is executed, the updated screen state is re-captured and analyzed by the perception module. The Agentic AI Logic module compares the expected outcome with the observed state to determine success, failure, or partial completion. Based on this evaluation, the system may proceed to the next step, retry the action, or re-generate a modified plan.

In addition, the system includes logging and monitoring mechanisms that record execution traces, decision paths, and evaluation results. This supports debugging, performance analysis, and traceability for research and testing purposes.

Table 4. 1: Technologies, techniques, architecture and algorithms used

Category	Description
Technologies	FastAPI, React, OCR (Tesseract), ESP32 HID, JSON-based communication
Techniques	Agentic decision-making, context management, feedback-driven evaluation, adaptive retry strategies
Algorithms	Perceive–Plan–Act–Evaluate loop, state validation, decision branching, retry and recovery logic
Architectures	Microservices architecture, agentic closed-loop system

Figure 4. 2: High-level layered architecture diagram

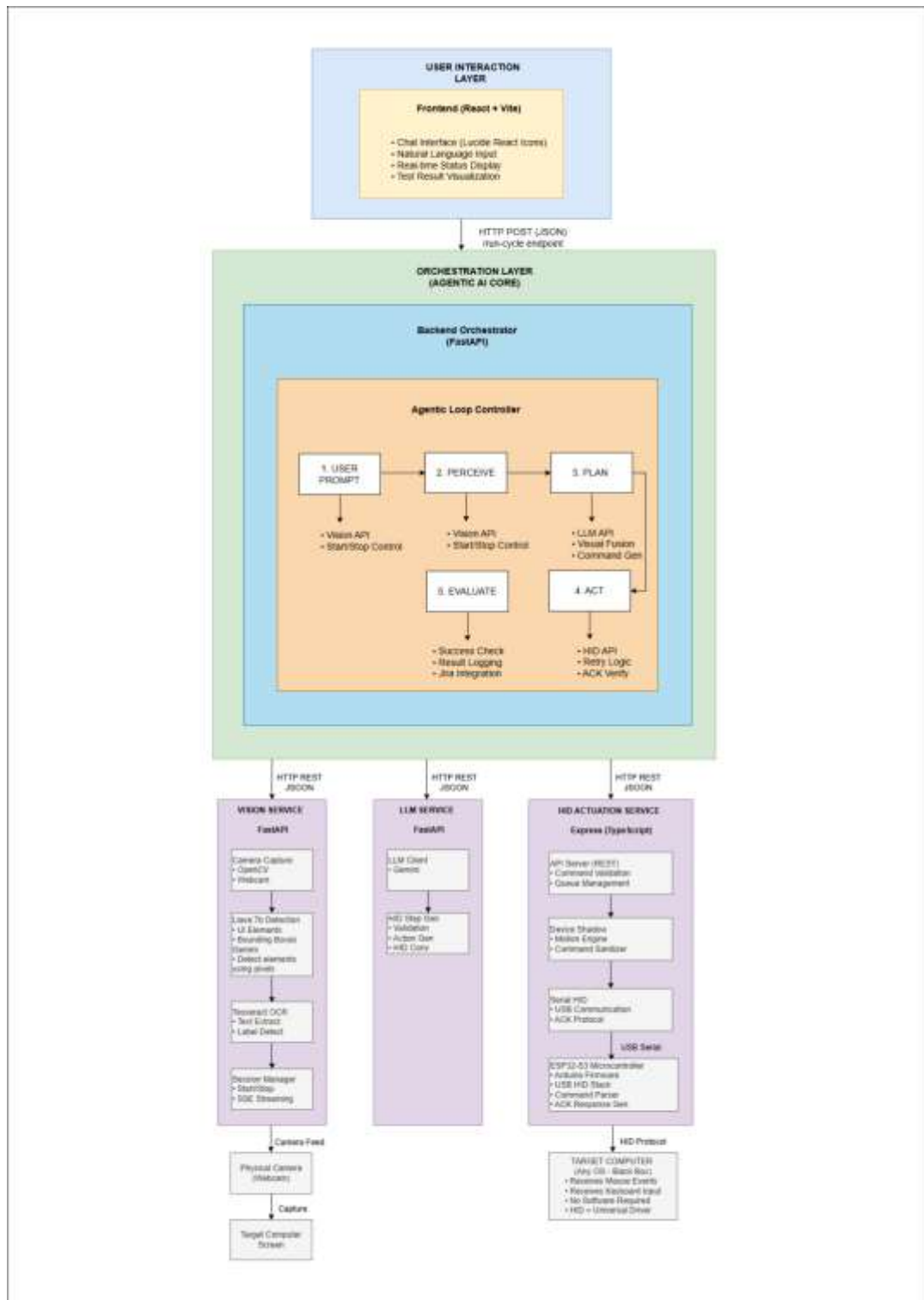


Figure 4.2 presents the layered structure of the system, where each layer represents a distinct functional stage of the automation pipeline. The perception layer is responsible for capturing and interpreting visual input. The reasoning layer handles instruction understanding and action planning using LLM-based models. The execution layer performs physical interactions through HID-based input emulation.

At the highest level, the Agentic AI Logic layer governs all interactions between these components. It ensures synchronization across perception, reasoning, and execution while maintaining task context and enabling continuous feedback-based decision-making. This layered design improves modularity, scalability, and robustness in real-world GUI automation scenarios.

4.2 Specific Component Description

This section describes the four core components of the system—**Vision (Visual Perception), LLM & Prompt Engineering, Agentic AI Logic, and HID Execution**—which collectively enable the closed-loop Agentic AI automation framework. Each module plays a distinct role in perception, reasoning, coordination, and execution to achieve autonomous GUI interaction.

Vision (Visual Perception) Module:

The Vision module is responsible for capturing and interpreting real-time graphical user interface (GUI) information from the target system using an external camera-based setup. It acts as the system’s sensory layer, enabling perception in black-box environments where internal system access is not available.

This component uses computer vision techniques and Optical Character Recognition (OCR) to extract structured UI elements such as buttons, input fields, labels, coordinates, and states from screen frames. The extracted data is transformed into a structured format that can be processed by downstream components.

The primary purpose of the Vision module is to provide accurate and context-rich environmental awareness to the Agentic AI Logic module. By continuously monitoring screen changes, it enables the system to understand the current interface state and supports real-time decision-making during task execution.

LLM & Prompt Engineering Module:

The LLM and Prompt Engineering module is responsible for converting natural language user instructions into structured and executable action plans. It serves as the reasoning layer of the system, bridging human intent and machine-level execution.

This module processes user inputs along with structured visual context received from the Vision module. Through carefully designed prompts and LLM-based reasoning, it generates step-by-step action sequences in a machine-readable format, including actions such as click, type, and navigation commands.

The generated outputs are structured in JSON format to ensure compatibility with downstream processing. Additionally, prompt engineering techniques are used to enforce output consistency, reduce ambiguity, and align generated actions with actual UI elements.

The module works closely with the Agentic AI Logic layer, which validates and refines these generated plans before execution.

Agentic AI Logic Module:

The Agentic AI Logic module is the central orchestration and decision-making unit of the system. It is responsible for managing the complete Perceive–Plan–Act–Evaluate cycle and ensuring coordinated interaction between all system components.

This module integrates inputs from the Vision module and the LLM-based planning module to maintain task context and determine the next best action. It continuously evaluates system state, monitors execution outcomes, and ensures that actions remain aligned with the user’s objective.

A key function of this module is adaptive decision-making. In cases where execution fails, UI elements are missing, or unexpected changes occur, the module dynamically adjusts its strategy. This may involve retrying actions, modifying execution paths, or triggering re-planning through the LLM module.

Additionally, it ensures consistency, reliability, and structured execution across all tasks by acting as the control layer that governs system behavior throughout the automation lifecycle.**HID Execution Module**

The HID (Human Interface Device) Execution module is responsible for translating structured action plans into real-world input events. It enables physical interaction with the target system by simulating keyboard and mouse operations through an ESP32-based hardware interface.

This module receives validated action commands from the Agentic AI Logic module and executes them in a precise sequence. Actions such as mouse movement, clicking, and text input are converted into low-level HID signals that interact directly with the operating system.

The primary purpose of the HID module is to ensure platform-independent execution of automation tasks without requiring internal system access, APIs, or application-level integration.

By bridging the gap between digital decisions and physical interaction, this module completes the execution phase of the agentic loop and enables real-world GUI automation.

4.2.1 Purpose

The purpose of the proposed system is to enable intelligent, context-aware, and feedback-driven GUI automation through the seamless integration of perception, reasoning, decision-making, and execution components.

The **Visual Perception component** is responsible for capturing and interpreting the user interface using camera-based input. It extracts structured information such as UI elements, labels, positions, and states using computer vision and OCR techniques. This enables the system to operate in black-box environments without relying on internal application access.

The **LLM & Prompt Engineering component** translates high-level natural language instructions into structured, machine-readable action plans. It bridges the gap between human intent and system execution by generating step-by-step actions aligned with the perceived UI context.

The **Agentic AI Logic component** acts as the central control unit of the system. It manages the overall workflow by coordinating perception, planning, execution, and evaluation. It maintains task context, validates decisions, and ensures that actions are aligned with both user intent and real-time interface conditions.

The **HID Actuation component** executes the planned actions by emulating human input through keyboard and mouse interactions using hardware-based control (e.g., ESP32). This enables non-intrusive interaction with the target system.

Collectively, these components enable adaptive, reliable, and human-like automation. The system supports dynamic decision-making, error handling, and continuous feedback integration, allowing it to function effectively in real-world, unpredictable GUI environments.

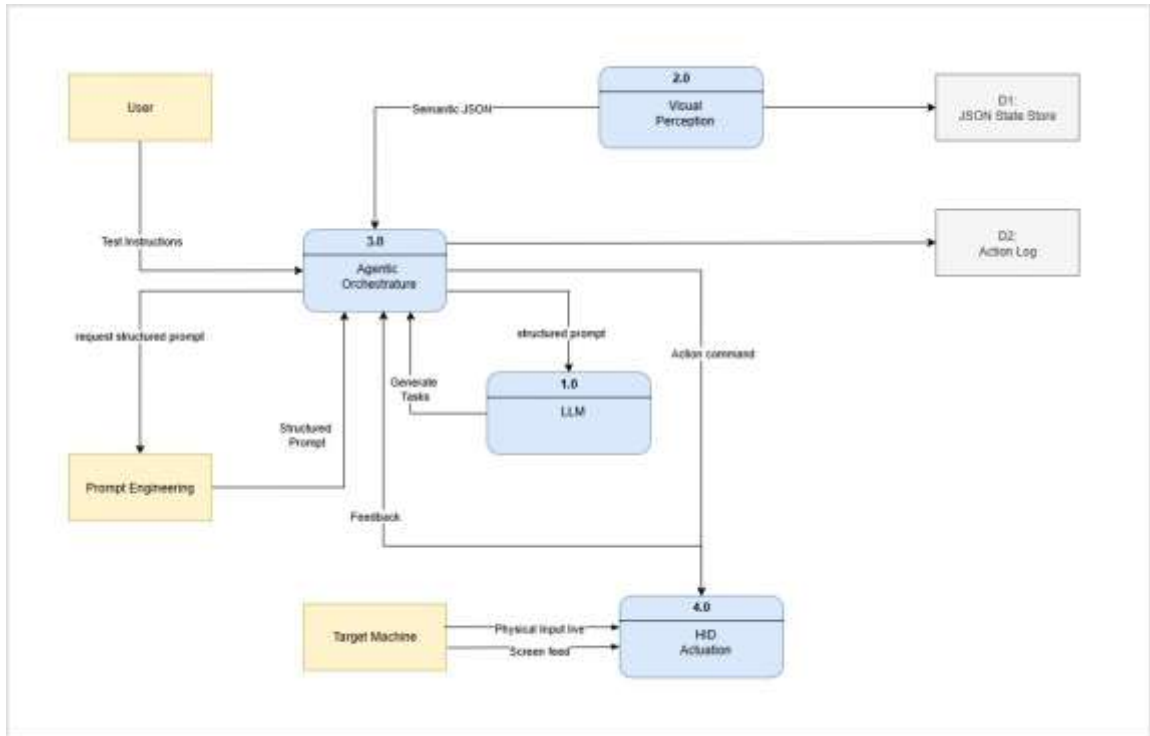


Figure 4. 3: component diagram

4.2.2 Workflow Overview

The system operates through a structured **Perceive–Plan–Act–Evaluate closed-loop workflow**, enabling continuous interaction between components and the environment.

Instruction Intake and Task Initialization:

The workflow begins when the user provides a high-level natural language instruction. The Agentic AI Logic initializes the task by storing the objective, defining execution boundaries, and maintaining an internal task state. This ensures that all subsequent actions remain aligned with the intended goal.

Perception and Context Acquisition:

The Visual Perception component captures the current screen using an external camera and processes it using OCR and UI detection techniques. It extracts structured data

such as UI elements, labels, coordinates, and states. This information is passed to the Agentic AI Logic to maintain an updated understanding of the environment.

Planning and Action Generation:

Based on the user instruction and perceived context, the LLM & Prompt Engineering component generates structured action steps. These steps are interpreted and refined by the Agentic AI Logic to ensure consistency with the current UI state and execution constraints.

Action Execution:

The validated action steps are translated into low-level commands and sent to the HID Actuation component. This component executes the actions using mouse movements, clicks, and keyboard inputs, enabling direct interaction with the system.

Evaluation and Feedback Integration:

After each action, the system captures the updated screen state through the perception module. The Agentic AI Logic compares the expected outcome with the observed result to determine whether the action was successful.

Adaptive Decision and Retry Mechanism:

If the evaluation detects discrepancies or failures, the Agentic AI Logic dynamically adjusts its strategy. This may involve retrying actions, modifying parameters, or requesting a revised plan from the LLM. This ensures robustness in handling dynamic and uncertain environments.

Exploratory Interaction Support:

In complex or ambiguous scenarios, the system supports exploratory behavior by selecting alternative actions based on available UI elements. This enables flexible and human-like interaction while maintaining alignment with the task objective.

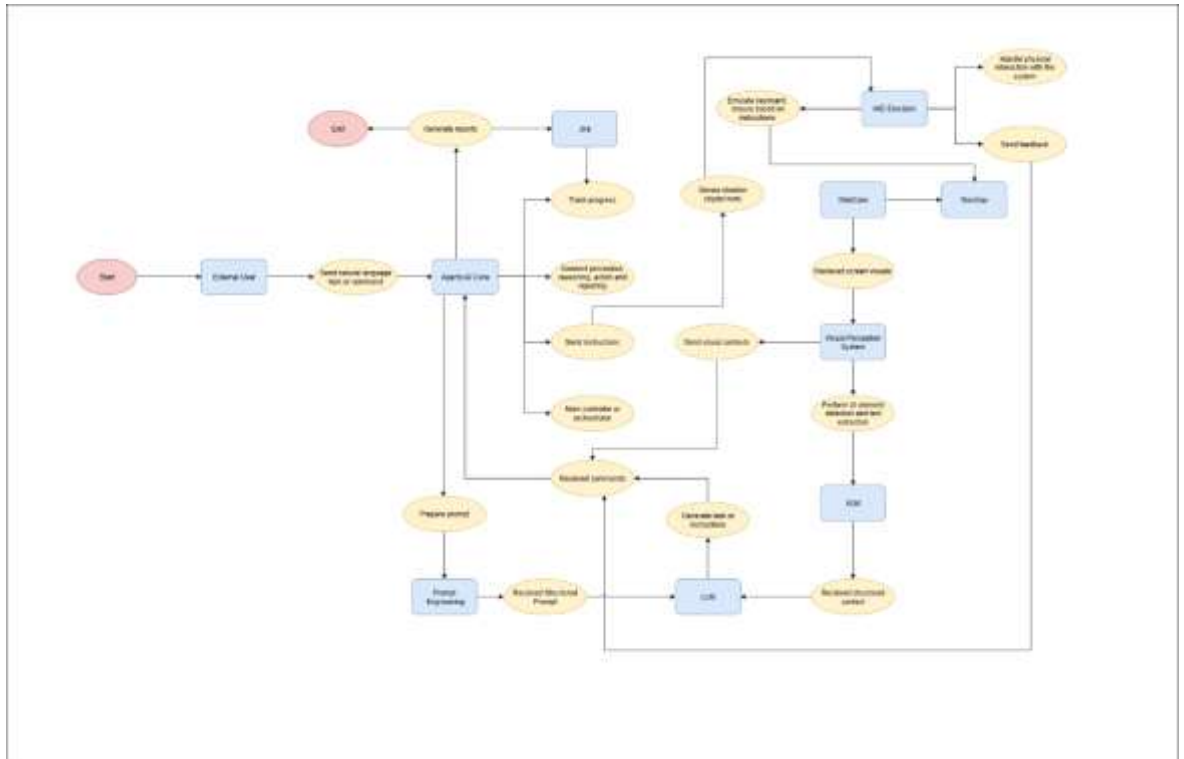


Figure 4. 4: Processing workflow

4.2.3 Input Processing

The system accepts natural language instructions through the user interface (e.g., “Enter username and login”). These inputs are processed collaboratively by multiple components.

Initially, the **Agentic AI Logic** normalizes the instruction by extracting the core task intent and defining execution context. This ensures consistency in how the task is represented internally.

The **LLM & Prompt Engineering component** then interprets the instruction and converts it into structured action steps. Prompt engineering techniques are applied to

reduce ambiguity and ensure that outputs are aligned with the system’s expected schema.

Simultaneously, the **Visual Perception component** provides real-time contextual data about the interface. This ensures that the generated actions are grounded in actual UI elements rather than assumptions.

By combining structured instruction understanding with real-time visual context, the system ensures that all inputs are transformed into reliable, executable, and context-aware actions. This integrated input processing approach enables accurate decision-making and improves overall system performance in dynamic environments.

4.2.4 Context Structuring and Decision Preparation

The system constructs a structured internal context by integrating inputs from multiple components, including user instructions, visual perception outputs, and execution constraints. This context serves as the foundation for all decision-making processes within the closed-loop workflow.

The **Visual Perception component** provides real-time UI data, including detected elements, positions, and states. This information is combined with the user’s instruction and current task progress maintained by the **Agentic AI Logic component**.

The **LLM & Prompt Engineering component** supports this process by utilizing prompt templates to structure reasoning tasks. These templates ensure that generated outputs remain consistent, machine-readable, and aligned with the current UI context. Rather than relying on static prompts, the system dynamically updates context based on continuous environmental feedback.

Through this integration, the system maintains a synchronized and up-to-date representation of the task environment, enabling accurate and context-aware decision preparation.

4.2.5 Action Planning and Decision Generation

Action planning is a collaborative process involving the **Agentic AI Logic** and the **LLM & Prompt Engineering component**.

The Agentic AI Logic determines the next best action based on task progress, historical execution data, and the current UI state. When higher-level reasoning is required, it invokes the LLM to generate structured action sequences.

The LLM decomposes complex instructions into smaller, executable steps such as clicking, typing, or selecting interface elements. These steps are generated in a structured format suitable for execution.

Before execution, all generated plans are validated and refined by the Agentic AI Logic to ensure alignment with the real-time UI context provided by the perception module. This ensures that decisions are both semantically correct and practically executable.

4.2.6 Perception-Driven Validation

The system employs a continuous perception-driven validation mechanism to ensure that all decisions are grounded in real-world interface conditions.

The **Visual Perception component** continuously captures and processes the screen, identifying UI elements using OCR and computer vision techniques. These detected elements are mapped against the targets specified in the planned actions.

This creates a bidirectional validation loop where:

- Perception data directly influences planning and decision-making
- Execution outcomes influence how future perception results are interpreted

The **Agentic AI Logic component** ensures that all actions are validated against the current UI state before and after execution. This prevents incorrect interactions caused by outdated or assumed interface conditions, significantly improving system reliability.

4.2.7 Execution and Error Handling

Once an action is validated, it is forwarded to the **HID Actuation component** for execution. This component translates structured action commands into low-level keyboard and mouse inputs using hardware-based emulation.

The execution process includes:

- Mouse movement and clicking
- Keyboard input and typing
- Sequential command execution

After each action, the **Visual Perception component** captures the updated UI state, and the **Agentic AI Logic** evaluates whether the expected outcome has been achieved.

If discrepancies or failures are detected, the system initiates adaptive recovery strategies, including:

- Retrying the action
- Adjusting execution parameters (e.g., coordinates, timing)
- Requesting revised plans from the LLM
- Re-evaluating the UI state before proceeding

This coordinated error-handling mechanism ensures robustness and resilience in dynamic and unpredictable environments.

4.2.8 Exploratory Branching

To support flexible and human-like interaction, the system enables exploratory branching during task execution.

The **Agentic AI Logic component** allows the system to deviate from strictly predefined action sequences when necessary. Based on real-time perception data, the system can dynamically select alternative interaction paths.

This includes:

- Exploring alternative UI elements when expected targets are unavailable
- Testing multiple interaction strategies for task completion
- Adapting behavior in response to unexpected interface states

The **LLM & Prompt Engineering component** assists in generating alternative strategies when required, while the **Perception component** ensures that all exploratory actions remain grounded in the current UI.

This capability enhances the system's flexibility and enables it to simulate human-like exploratory testing behavior while maintaining goal-oriented execution.

4.2.9 Optimization and Evaluation

System performance is continuously monitored and evaluated using execution logs and feedback collected during runtime.

The **Agentic AI Logic component** aggregates data from all components, including perception outputs, planning decisions, execution results, and error logs. This data is used to assess system performance and refine decision-making strategies.

Evaluation metrics include:

- Task completion success rate

- Accuracy of action execution
- Alignment between planned and executed actions
- Adaptability to UI changes
- Effectiveness of error recovery mechanisms
- Reduction of redundant or failed actions

The **LLM component** can be further optimized through improved prompt design, while the **Perception component** can be refined to enhance detection accuracy. Similarly, the **HID Actuation component** can be tuned to improve execution precision and timing.

This continuous optimization process strengthens the overall robustness and efficiency of the system, enabling reliable operation in real-world black-box environments.

4.3 Agentic Decision-Making and Action Planning Framework

This section presents the methodology used by the overall system for coordinating perception, reasoning, planning, and execution within a unified agentic framework. The objective is to enable the reliable transformation of high-level natural language instructions into validated and executable UI actions while maintaining continuous awareness of dynamic interface states.

The system adopts a **closed-loop decision-making architecture**, where all four components—Visual Perception, LLM & Prompt Engineering, Agentic AI Logic, and HID Actuation—collaborate to achieve adaptive and context-aware automation. While the LLM contributes reasoning capabilities, the Agentic AI Logic remains the central controller responsible for validation, coordination, and execution management.

4.3.1 Instruction Processing and Context Initialization

The process begins with the user providing a natural language instruction. This instruction is received and normalized by the **Agentic AI Logic component**, which extracts the core task objective and removes ambiguity.

At the same time, the **Visual Perception component** provides the current UI state by detecting interface elements, positions, and attributes. This perception data is combined with the user instruction, previously executed actions, and system constraints to construct a structured task context.

This unified context serves as the foundation for all subsequent reasoning, planning, and execution decisions across the system.

4.3.2 Context-Aware Reasoning Strategy

The system employs a context-aware reasoning strategy where decision-making is guided by both semantic understanding and real-time UI data.

The **LLM & Prompt Engineering component** is used as a reasoning assistant to generate potential action strategies when required. Structured prompt templates are used to ensure that outputs remain consistent, interpretable, and aligned with system requirements.

However, all reasoning outputs are processed and validated by the **Agentic AI Logic component**, ensuring that generated actions are grounded in the actual UI context provided by the perception module. This prevents the system from executing actions based on incorrect assumptions or outdated information.

4.3.3 Action Decomposition and Planning

High-level user objectives are decomposed into smaller, atomic actions through a collaborative process between the **Agentic AI Logic** and the **LLM component**.

Each task is broken down into individual UI interactions such as clicking, typing, or selecting elements. The LLM may assist in generating these step sequences, especially for complex or ambiguous instructions.

The **Agentic AI Logic component** evaluates each proposed action against the real-time UI state provided by the perception module. Only actions that are feasible and contextually valid are selected for execution. This ensures that planning remains adaptive and robust in dynamic environments.

4.3.4 Structured Action Representation

Once validated, actions are converted into a structured format suitable for execution and inter-component communication.

Each action is represented using a standardized schema (e.g., JSON), including:

- Action type (click, type, move, etc.)
- Target UI element
- Parameters (coordinates, input text, etc.)
- Expected outcome

This structured representation enables seamless communication between the **Agentic AI Logic** and the **HID Actuation component**, ensuring precise and deterministic execution of actions.

4.3.5 Action Validation and Safety Checks

Before execution, the system performs multiple validation checks to ensure safety and correctness.

The **Agentic AI Logic component**, supported by perception data, verifies:

- Alignment between action targets and detected UI elements
- Structural correctness of the action schema
- Consistency with the overall task objective

- Feasibility within the current UI state

If inconsistencies or invalid conditions are detected, the system rejects or refines the action plan. The **LLM component** may be re-invoked with updated context to generate alternative strategies.

This validation layer acts as a critical safeguard, preventing incorrect or unsafe actions from being executed.

4.3.6 Feedback-Driven Adaptation

After execution, the **HID Actuation component** performs the action through keyboard and mouse emulation. The system then captures the updated UI state using the **Visual Perception component**.

The **Agentic AI Logic** compares the observed outcome with the expected result defined in the action plan. If mismatches are detected, the system initiates adaptive recovery mechanisms, including:

- Re-planning the next step
- Modifying execution parameters
- Updating the task context
- Requesting alternative action paths from the LLM

This feedback-driven adaptation ensures continuous correction and enables the system to handle dynamic UI changes, delays, and execution uncertainties effectively.

4.3.7 Continuous Optimization of Decision Strategy

The system incorporates a continuous optimization mechanism based on execution logs and feedback collected during operation.

The **Agentic AI Logic component** analyzes performance metrics such as:

- Task success rate
- Action execution accuracy
- Frequency of failures and retries
- Efficiency of decision-making

Insights derived from these metrics are used to refine decision strategies, improve prompt design in the **LLM component**, enhance perception accuracy, and optimize execution timing in the **HID component**.

This feedback-driven refinement process improves long-term system robustness, reduces unnecessary re-planning, and enhances adaptability in diverse and unpredictable GUI environments.

4.4 Design System Diagrams

System design diagrams were developed to identify the essential components and structure the implementation of the proposed model. Figure 4.4 illustrates the Data Flow Diagram (DFD), which represents the flow of data between the system components and supports a clear understanding of system interactions. This diagram was used as a reference during the design and implementation of the proposed research component.

4.4.1 DFD Diagram

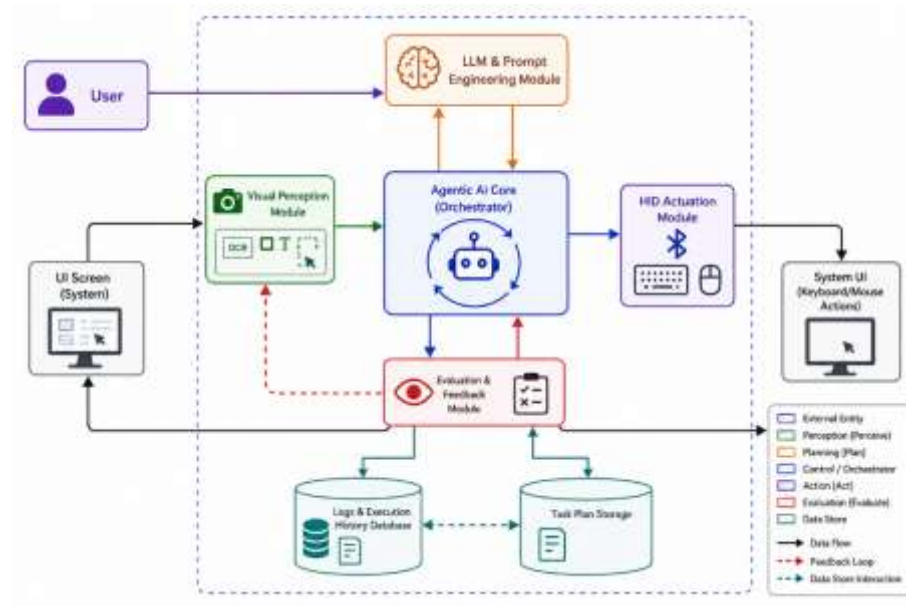


Figure 4. 5: DFD diagram

5. IMPLEMENTATION AND TESTING

5.1 Implementation

The implementation focuses on a fully integrated, agentic, screen-aware automation system that combines perception, reasoning, decision-making, and execution into a unified workflow. The system is designed around a continuous **Perceive–Plan–Act–Evaluate loop**, enabling adaptive and context-aware GUI automation in black-box environments.

Unlike traditional automation approaches that rely on static scripts or predefined workflows, the proposed system operates dynamically by continuously updating its internal state based on real-time feedback. The **Agentic AI Logic component** acts as the central control unit, coordinating all other components to ensure consistent and reliable task execution.

User instructions are processed and interpreted through the **LLM & Prompt Engineering component**, which converts natural language inputs into structured action plans. These plans are not executed directly; instead, they are passed to the Agentic AI Logic for validation and contextual alignment.

The **Visual Perception component** continuously captures the screen using an external camera and extracts structured UI information such as element labels, types, coordinates, and states. This perception data is used to ground all decisions in the actual interface environment.

The **HID Actuation component** executes validated actions by emulating keyboard and mouse inputs through hardware-based interaction. This allows the system to interact with the target application in a non-intrusive manner, without requiring APIs or internal system access.

A key aspect of the implementation is the integration of **feedback-driven adaptation**. After each action is executed, the system captures the updated UI state and evaluates whether the intended outcome has been achieved. Based on this evaluation, the Agentic AI Logic determines whether to proceed, retry, or re-plan the next step.

This coordinated interaction between perception, reasoning, decision-making, and execution enables robust operation in dynamic and unpredictable GUI environments, ensuring that the system remains adaptive, reliable, and context-aware throughout the task lifecycle.

5.1.1 System Component Overview

The system is composed of four primary components that operate collaboratively within the agentic control loop.

The **Visual Perception component** is responsible for capturing and interpreting the graphical user interface. It utilizes computer vision and OCR techniques to detect UI elements and convert them into structured data. This enables the system to understand the environment without relying on internal application access.

The **LLM & Prompt Engineering component** functions as the reasoning layer of the system. It processes natural language instructions and generates structured action plans using prompt engineering techniques. These outputs are designed to be machine-readable and aligned with the system's execution requirements.

The **Agentic AI Logic component** serves as the central orchestration and decision-making engine. It manages the complete execution lifecycle, including context initialization, action validation, task progression, and feedback-based adaptation. It ensures that all actions are consistent with the current UI state and overall task objective.

The **HID Actuation component** handles the execution of actions by simulating keyboard and mouse inputs through hardware-based interaction. It receives structured commands from the Agentic AI Logic and performs precise, real-time interactions with the target system.

Together, these components form a cohesive system that supports end-to-end automation, from instruction understanding to physical execution.

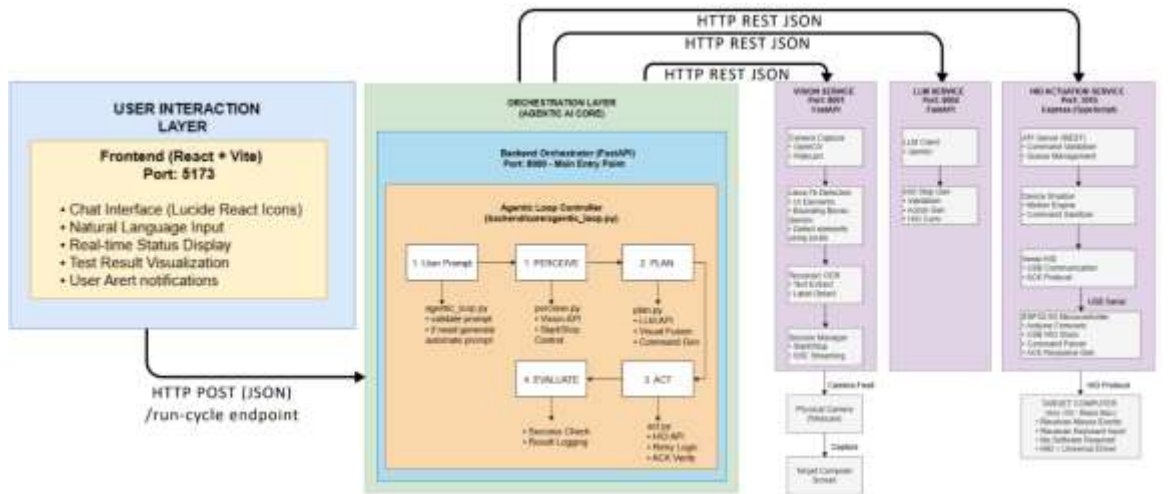


Figure 5.1: Endpoint flow diagram

5.1.2 Instruction-to-Action Coordination Pipeline

The system implements an instruction-to-action coordination pipeline that transforms high-level user tasks into structured and executable behaviors within a closed-loop architecture. This pipeline is not a simple linear transformation; instead, it is a coordinated process involving perception, reasoning, decision-making, execution, and evaluation.

When a user task is received, the **Agentic AI Logic component** first determines whether visual perception is required using a decision function. This step ensures efficiency by avoiding unnecessary perception cycles for tasks that do not depend on UI context.

If visual input is required, the **Visual Perception component** is activated to capture the current screen state and extract structured UI data. This includes element labels, positions, and states, which are essential for grounding subsequent decisions.

The extracted perception data, along with the user instruction, is then passed to the **LLM & Prompt Engineering component**, which generates structured action plans. These plans represent candidate execution strategies rather than final commands.

The **Agentic AI Logic component** evaluates and refines these plans to ensure alignment with the real-time UI context and task objective. Only validated actions are forwarded to the execution layer.

This coordinated pipeline ensures that perception, reasoning, and execution remain tightly integrated while maintaining centralized control over decision-making.

```

# Plan (Pass perception OR None)
logging.info("🟡 Planning action...")
action_plan = plan(user_task=user_task, perception=perception)
save_json(action_plan, "action_plan", run_folder)
logging.info(
    "Generated Action Plan:\n%s",
    json.dumps(action_plan, indent=2)
)

validation = action_plan.get("validation", {})

```

Figure 5.2: Agentic instruction-to-action coordination flow

5.1.3 Visual Context Construction and State Validation

Before executing any action, the system constructs a structured representation of the current environment using outputs from the **Visual Perception component**.

This representation includes:

- Detected UI elements
- Element positions and attributes
- Screen state metadata

The **Agentic AI Logic component** maintains a baseline UI state and continuously compares it with updated perception outputs after each action. A comparison mechanism identifies differences such as newly appeared elements, removed components, or state changes.

This validation process ensures that:

- Actions are grounded in real-time observations
- Expected outcomes are verified accurately
- Incorrect or outdated assumptions are avoided

By integrating perception-driven validation, the system maintains high reliability in dynamic GUI environments where interface states frequently change.

5.1.4 Execution and HID Control Layer

Once an action plan is validated, execution is carried out by the **HID Actuation component**, which translates high-level actions into low-level hardware instructions.

The execution process includes:

- Mouse movement and click actions
- Keyboard input and text entry
- Sequential command execution

These commands are executed through hardware-based interaction (e.g., ESP32), allowing the system to interact with the target interface as a human user.

The **Agentic AI Logic component** manages execution through a retry mechanism (e.g., `act_with_retry`), which attempts to recover from transient failures. If an action fails, the system:

- Re-evaluates the UI state using the perception module
- Adjusts execution parameters if necessary
- Retries the action within a defined limit

After execution, the updated screen state is captured and stored. This ensures traceability and supports debugging, performance analysis, and future optimization.



```
# Act
logging.info(" 📌 Acting on plan...")
action_result = act_with_retry(action_plan, max_retries=3)
save_json(action_result, "action_result", run_folder)
logging.debug(f"Action Result: {action_result}")

latest_perception = None
latest_screens = []
```

Figure 5.3 : HID execution and retry mechanism flow

5.1.5 Feedback Loop and Evaluation Mechanism

The system incorporates a continuous feedback loop to evaluate performance after each action execution.

After execution, the **Visual Perception component** captures the updated UI state, and the **Agentic AI Logic component** compares the observed outcome with the expected result defined in the action plan.

Based on this evaluation:

- If the outcome matches expectations, the system proceeds to the next step or completes the task
- If a failure or mismatch is detected, adaptive recovery strategies are triggered

These strategies may include:

- Re-planning using the **LLM & Prompt Engineering component**
- Modifying execution parameters
- Retrying actions
- Updating task context

In addition, the system provides user-level feedback (e.g., success or failure notifications), improving usability and transparency during execution.

The final evaluation output is structured and includes:

- Vision usage status
- Generated action plan
- Execution results
- Evaluation status
- UI state change summary

This structured feedback ensures full traceability of the agentic decision cycle and supports detailed analysis of system performance.

5.2 Testing

Testing was conducted to evaluate the correctness, robustness, and operational stability of the proposed agentic GUI automation system. The evaluation covered the complete **Perceive–Plan–Act–Evaluate** cycle, incorporating all four major components: visual perception, LLM-based planning, agentic decision-making, and HID-based execution.

Unlike traditional automation systems that follow a linear execution model, the proposed system was tested as a **closed-loop agentic framework**, where each action is continuously validated using real-time visual feedback before proceeding. This ensures not only functional correctness but also adaptive behavior under dynamic and uncertain GUI conditions.

The testing scope included the following system functionalities:

- Vision activation and decision logic (whether perception is required)
- Screen perception and UI element extraction (Vision module)
- Natural language interpretation and structured plan generation (LLM & Prompt Engineering module)
- Agentic AI coordination and decision-making across task states
- HID-based execution of actions (keyboard/mouse emulation)
- Retry mechanisms and failure recovery strategies
- Feedback-driven evaluation and screen state comparison
- End-to-end task execution and completion behavior

In addition to functional validation, non-functional aspects such as **execution latency, responsiveness, reliability, and stability of the feedback loop** were also evaluated.

5.2.1 Test Plan and Strategy

A structured testing strategy was adopted to evaluate the integrated performance of all system components under realistic GUI automation scenarios. The focus was on validating both **component-level functionality** and **end-to-end coordination** within the agentic loop.

Testing was organized around the following core components:

- **Visual Perception Module**
 - Screen capture accuracy
 - OCR-based text extraction
 - UI element detection and coordinate mapping
- **LLM & Prompt Engineering Module**
 - Natural language understanding
 - Structured action plan generation
 - Consistency and completeness of generated steps
- **Agentic AI Logic Module**
 - Task state management
 - Context-aware decision-making
 - Coordination between perception, planning, and execution
 - Adaptive retry and re-planning behavior

- **HID Actuation Module**
 - Accuracy of mouse and keyboard simulation
 - Execution timing and synchronization
 - Reliability under repeated actions

Priority was given to **end-to-end execution scenarios**, where a complete user instruction flows through perception, planning, execution, and evaluation. This ensured that all components work cohesively within the closed-loop system.

5.2.2 Test Case Design

The following test cases were designed to evaluate the integrated performance, reliability, and adaptive decision-making capability of the proposed agentic GUI automation system. The evaluation focuses on the complete **Perceive–Plan–Act–Evaluate loop**, ensuring proper coordination between the visual perception module, LLM-based planning module, agentic decision-making logic, and HID execution layer.

Each test case targets a specific component as well as overall system behavior, enabling both **component-level validation** and **end-to-end system evaluation**.

Table 5. 1: End-to-End Login Task Execution Using Agentic Loop

Test Case ID	01
Test Case	Verify full login task execution using the integrated agentic system
Test Scenario	Validate whether the system can complete a login workflow using perception, planning, decision-making, and HID execution

Input	Natural language instruction with visual screen data containing Username, Password, and LOGIN button
Expected Output	1. Vision module activates correctly 2. UI elements detected with valid perception output 3. LLM generates structured step plan 4. Agentic AI coordinates execution flow 5. HID executes commands successfully 6. Final evaluation returns success
Actual Result	1. Vision activated successfully 2. Perception detected Username, Password, LOGIN button 3. Structured step plan generated with HID commands 4. Execution attempted with retries (3 attempts) 5. Execution failed due to evaluation/parsing issue 6. Final report returned success = false
Status	Fail

Table 5. 2: Visual Perception and UI Element Detection

Test Case ID	02
Test Case	Verify screen perception and UI element extraction
Test Scenario	Validate accuracy of UI detection from camera-based visual input

Input	Login screen with Username, Password, LOGIN button
Expected Output	All required UI elements detected with coordinates and confidence scores
Actual Result	Username, Password, LOGIN button, and additional UI elements (e.g., system icons) detected with confidence levels (~0.8–0.9)
Status	Pass

Table 5. 3: LLM-Based Task Planning and Action Generation

Test Case ID	03
Test Case	Verify natural language understanding and structured plan generation
Test Scenario	Validate whether the system converts user instruction into structured action steps
Input	"Enter username 'tharushi', enter password '123', click LOGIN button"
Expected Output	Structured step plan with ordered actions and mapped HID commands

Actual Result	Step plan generated with clear reasoning and action sequence including mouse movement, click, and text input
Status	Pass

Table 5. 4: HID Execution and Retry Mechanism

Test Case ID	04
Test Case	Verify HID execution reliability and retry logic
Test Scenario	Validate execution of low-level commands and system recovery on failure
Input	Structured step plan with login actions
Expected Output	HID commands executed successfully with final status = success
Actual Result	Execution attempted multiple times (3 retries), failure occurred due to evaluation/parsing issue, final status remained unsuccessful
Status	Fail

Table 5. 5: Feedback-Driven Evaluation and Recovery Mechanism

Test Case ID	05
Test Case	Verify evaluation logic and recovery recommendations

Test Scenario	Validate whether the system correctly analyzes failures and suggests improvements
Input	Failed execution scenario from login workflow
Expected Output	System should generate failure analysis, step-level feedback, and recovery suggestions
Actual Result	Final report generated with failure reason, step analysis, and recommendations (e.g., HID permissions, window focus, step decomposition improvements)
Status	Pass

6. RESULTS AND DISCUSSIONS

6.1 Results

This section presents the empirical outcomes of implementing and evaluating the proposed agentic GUI automation system. The evaluation focuses on assessing whether natural language instructions can be reliably transformed into structured, executable user interface actions through an integrated **Perceive–Plan–Act–Evaluate** framework. The system combines four key components: visual perception, LLM-based planning and prompt engineering, agentic decision-making, and HID-based execution.

The evaluation was conducted using real execution runs generated by the system, where each run represents a complete automation cycle initiated by a natural language instruction.

During each cycle, the system performs screen perception, interprets the interface state, generates structured action plans, executes commands through hardware-based interaction, and evaluates the outcome using feedback from the environment. All stages of execution are recorded as structured JSON logs, capturing detailed information about perception results, planning outputs, execution steps, retry attempts, and final evaluation reports.

Unlike traditional automation approaches that rely on APIs or internal application access, the system was evaluated in a **black-box environment**, where all decisions are based solely on external visual input and coordinated agentic control. This allows for a more realistic assessment of system behavior in practical scenarios where internal system access is limited or unavailable.

The test scenarios were designed to reflect a range of real-world conditions, including:

- Successful task execution workflows (e.g., login and navigation tasks)
- Multi-step interaction sequences requiring coordination across components
- Dynamic interface conditions with changing UI states
- Execution failure scenarios requiring retry and recovery mechanisms
- Cases with partial or ambiguous visual information

This diverse set of scenarios enabled evaluation of both **component-level performance** and **end-to-end system behavior** under varying levels of complexity and uncertainty.

The results indicate that the system is capable of effectively interpreting high-level user instructions and transforming them into structured, machine-executable action plans. The **visual perception module** successfully extracts UI elements and contextual information from screen inputs, providing a reliable foundation for decision-making. The **LLM and prompt engineering module** demonstrates strong capability in generating coherent and structured action sequences aligned with user intent. The **Agentic AI Logic module** effectively coordinates the workflow by maintaining task context, validating decisions, and managing execution flow. The **HID module** enables practical interaction with the target system by executing low-level input commands.

Furthermore, the system demonstrates adaptive behavior through its feedback-driven control loop. By continuously evaluating execution outcomes using updated visual input, the agent can detect failures, initiate retries, and adjust its strategy when necessary. This highlights the system’s ability to operate beyond static automation and perform context-aware decision-making in dynamic environments.

However, the results also reveal certain limitations. While individual components perform reliably, challenges arise in **end-to-end execution stability**, particularly in complex multi-step tasks. Issues such as execution failures, retry inconsistencies, and evaluation parsing errors can impact overall task completion. These observations indicate that while the architecture is functionally sound, further improvements are required in integration robustness, error handling, and execution synchronization.

Overall, the findings confirm the feasibility of integrating visual perception, LLM-based reasoning, agentic decision-making, and hardware-based execution into a unified automation framework. The system demonstrates strong potential for real-world GUI automation in black-box environments, while also highlighting key areas for future enhancement in reliability and performance.

6.1.1 Overall Execution Outcomes

To evaluate the proposed agentic GUI automation system, an end-to-end execution run was conducted using a real-world login automation scenario. Unlike traditional automation approaches that depend on external tools or API-based request handling, the system operates through an internal **closed-loop agentic pipeline**, where a single natural language instruction triggers the complete **Perceive → Plan → Act → Evaluate** cycle across all system components.

Figure 6.1 illustrates the structured JSON output generated during a full execution run. This JSON file serves as the primary evaluation artifact, capturing detailed information from all four components, including visual perception results, LLM-generated plans,

agentic decision-making steps, HID execution commands, and final evaluation outcomes.

```
agent_outputs > run_20260418_164056 > {} full_cycle.json > {} perception > {} vision_data > {} image_size > ## 1
1  {
2    "run_id": "61e22a16-7299-4fd7-aa16-93cda0718cb9",
3    "user_task": "Enter username 'tharushi' in the Username field, enter password '123' in the Password",
4    "started_at": "2026-04-18T11:10:56.937159+00:00",
5    "finished_at": "2026-04-18T11:15:55.974048+00:00",
6    "vision_used": true,
7    "perception": {
8      "status": "completed",
9      "session_id": "shot_1776510666_c888e6",
10     "source_frame": "D:\\researchRepo\\ScreenAwareTaskAgent\\vision\\data\\sessions\\shot_1776510666_
11     "processed_at": 1776510722.859908,
12     "final_json_path": "D:\\researchRepo\\ScreenAwareTaskAgent\\vision\\data\\sessions\\shot_17765106
13     "vision_data": {
14       "image": "image_0.png",
15       "image_size": [
16         640,
17         480
18       ],
19       "coordinate_system": "pixel",
20       "elements": {
21         {
22           "id": "0",
23           "type": "input",
24           "label": "Username",
25           "description": "Input field for entering username",
26           "state": "empty",
27           "dx": 368,
28           "dy": 205,
29           "confidence": 0.9,
```

Figure 6.1: Agentic system execution output (run JSON file)

The system was evaluated using the following instruction:

“Enter username ‘tharushi’ in the Username field, enter password ‘123’ in the Password field, then click the Login button to verify the login screen.”

The execution was performed using the full system pipeline with visual perception enabled. The system successfully captured the screen state using the perception module and identified relevant UI elements such as the Username field, Password field, and LOGIN button with high confidence.

Perception and Context Extraction:

The **visual perception module** generated a structured representation of the screen environment, which was stored within the JSON output. The detected UI elements included:

- Username input field (dx: 368, dy: 205)
- Password input field (dx: 368, dy: 265)
- LOGIN button (dx: 368, dy: 310)

In addition to task-relevant elements, the system also detected auxiliary interface components such as system icons (e.g., Start menu, browser icons), demonstrating broader environmental awareness.

This structured perception data formed the foundation for decision-making by providing real-time contextual grounding for the agentic control logic.

Action Planning and Decision Coordination:

Using the perception output and user instruction, the **LLM and prompt engineering module** generated a structured action plan, which was then interpreted and validated by the **Agentic AI Logic module**.

The high-level plan included:

- Focus on Username field → enter username
- Focus on Password field → enter password
- Click LOGIN button

These steps were further decomposed into low-level executable actions, including:

- mouse_move
- mouse_click

- type_text

This demonstrates the system's ability to transform semantic user intent into structured, machine-readable action sequences. The Agentic AI Logic ensured that all planned actions were aligned with the current UI state before proceeding to execution.

Execution Behavior and Feedback Loop:

The validated action plan was forwarded to the **HID execution module**, which performed physical interaction with the system using keyboard and mouse emulation.

During execution, the system attempted to complete the login workflow. The execution process included retry logic managed by the Agentic AI Logic module. The results were recorded in the structured execution logs.

Key observations:

- Execution attempts: 3
- Final execution status: unsuccessful
- Issue encountered: evaluation-stage inconsistency due to invalid or incomplete response parsing

Despite successful perception and planning, the execution stage experienced failure due to issues in evaluation and retry handling. The system detected this discrepancy through its feedback loop and attempted recovery, demonstrating adaptive behavior.

Evaluation and Final Outcome:

The **evaluation module**, coordinated by the Agentic AI Logic, analyzed the execution outcome by comparing expected results with the observed UI state.

The final report generated by the system indicated:

- Overall success: false

- Steps completed: 0
- Steps failed: 1
- Identified issue: interruption in execution sequence
- Suggested recommendations:
 - Ensure proper HID permissions
 - Maintain active window focus
 - Decompose complex steps into smaller actions

This structured reporting mechanism highlights the system’s transparency and ability to provide meaningful feedback for debugging and improvement.

Key Observations from Execution Results:

The experiment demonstrates that the integrated system is capable of:

- Converting natural language instructions into structured execution plans (LLM module)
- Extracting real-time UI context using visual perception (Vision module)
- Coordinating decisions through a centralized control loop (Agentic AI Logic)
- Executing actions using hardware-level interaction (HID module)
- Maintaining a feedback-driven loop for validation and recovery

However, the results also reveal a critical limitation in **execution reliability**, particularly under retry conditions and evaluation-stage inconsistencies. While the architecture successfully integrates all components, further improvements are required in execution synchronization, error handling, and evaluation robustness to achieve consistent end-to-end success.

6.1.2 Quantitative Performance Summary

The experimental evaluation of the proposed agentic automation system revealed measurable variations in execution time and system behavior depending on instruction complexity, visual clarity, and the level of reasoning required across the integrated perception–planning–execution pipeline. Since the system operates as a coordinated multi-component architecture, performance is influenced by the interaction between the visual perception module, LLM-based planning, Agentic AI decision-making, and HID-based execution.

For deterministic tasks where UI elements were clearly detected by the visual perception module and directly aligned with user instructions, the system demonstrated efficient performance with low latency. In these scenarios, the perception component accurately extracted structured UI data, the LLM generated concise action plans with minimal ambiguity, and the Agentic AI controller executed decisions with limited need for re-evaluation. This resulted in fast and stable end-to-end execution cycles.

In contrast, multi-step workflows involving sequential UI interactions introduced moderate latency due to increased coordination overhead among system components. The Agentic AI module was required to maintain task state across multiple steps, while the perception module continuously updated UI context after each action. Additionally, the LLM planning component contributed to processing time when generating intermediate steps for complex workflows such as navigation, form filling, and confirmation actions. As a result, execution time increased proportionally with the number of atomic actions and validation cycles.

Higher latency and variability were observed in scenarios involving ambiguous instructions or incomplete visual context. In such cases, the LLM required additional reasoning cycles to infer user intent, while the Agentic AI module performed iterative decision-making supported by feedback from the perception module. The need for re-planning, combined with retry mechanisms in the HID execution layer, significantly increased overall execution time. However, this additional processing improved robustness and allowed the system to handle uncertainty more effectively.

The performance behavior can therefore be categorized into three patterns. Low latency is observed in visually grounded and well-defined tasks. Moderate latency occurs in structured, multi-step workflows requiring coordination across components. High latency appears in ambiguous or failure-prone scenarios where iterative reasoning, perception updates, and retry mechanisms are required.

These findings highlight that execution time in the system is dynamically influenced by the interaction between perception accuracy, reasoning complexity, and execution stability. While this introduces variability compared to traditional rule-based automation, it significantly enhances adaptability and robustness in real-world black-box environments. The trade-off between execution efficiency and intelligent decision-making is therefore a fundamental characteristic of the proposed architecture.

6.1.3 Accuracy and Output Structure Quality

A key outcome of this study is the system’s ability to generate consistent, structured, and machine-readable outputs across all stages of the automation pipeline. The integrated architecture produces a comprehensive execution trace in JSON format, capturing outputs from all four components: visual perception data, LLM-generated plans, Agentic AI decision states, and HID execution logs. This structured representation ensures full traceability from initial instruction to final execution outcome.

The quality of system outputs can be analyzed across three main dimensions: semantic accuracy, contextual grounding, and execution reliability.

Semantic Accuracy:

The system demonstrated strong capability in translating natural language instructions into structured task flows. The LLM and prompt engineering module effectively interpreted both explicit and moderately ambiguous instructions, generating coherent action sequences. These outputs were further refined by the Agentic AI module, which ensured that generated plans aligned with task objectives and execution constraints. This layered reasoning approach improved the overall correctness of task decomposition.

Contextual Grounding:

A significant strength of the system lies in its reliance on real-time visual perception for decision-making. The perception module consistently extracted UI elements, including text labels, coordinates, and interface structure, using OCR and computer vision techniques. The Agentic AI module utilized this structured data to validate and adjust action plans generated by the LLM. As a result, all actions were grounded in the actual UI state, reducing the likelihood of invalid or non-existent interactions and improving system reliability in dynamic environments.

Execution Reliability:

The system successfully transformed high-level plans into low-level HID command sequences, enabling real-world interaction with the target interface. The HID module executed actions such as mouse movement, clicking, and text input based on structured instructions. Additionally, the inclusion of retry mechanisms and feedback-based evaluation allowed the system to detect and recover from execution failures. The Agentic AI module played a critical role in this process by analyzing execution outcomes and coordinating re-planning when necessary.

The integration of these components ensures that the system produces not only interpretable outputs but also directly executable action pipelines. The structured JSON output includes detailed logs of perception results, decision-making steps, action sequences, execution attempts, and evaluation outcomes, enabling comprehensive debugging and performance analysis.

Overall, the results demonstrate that the proposed system effectively bridges the gap between high-level intent understanding and low-level execution. By combining visual perception, LLM reasoning, agentic coordination, and hardware-based interaction, the system achieves both practical applicability and high transparency in black-box GUI automation environments.

Semantic Accuracy:

The system demonstrated strong capability in interpreting high-level natural language instructions into structured task flows. For example, login-related tasks were consistently decomposed into ordered steps such as focusing the username field, entering credentials, switching to the password field, and triggering the login action. This indicates that the Agentic AI controller effectively translates user intent into meaningful procedural logic using LLM-driven reasoning combined with prompt engineering strategies.

Contextual Grounding:

Generated actions showed strong alignment with real-time visual perception data. The system consistently utilized UI elements detected by the vision module, including coordinates, labels, and element states, to ground decision-making. This prevented the generation of invalid or non-existent UI interactions and ensured that all actions were based on actual screen state rather than inferred assumptions. The integration between perception output and planning logic significantly improved decision reliability in dynamic environments.

Execution Reliability:

The final output, particularly in the form of HID command sequences and step-level execution logs, demonstrated that the system can produce directly executable instructions for hardware-level interaction. Each action was converted into low-level commands such as mouse movement, click events, and text input operations, enabling real-world interaction through the ESP32-based HID layer. Additionally, the inclusion of retry logic and step-level evaluation within the JSON run file provides an additional layer of reliability by enabling recovery from partial execution failures.

These results are significant because the Agentic AI system does not only generate descriptive or intermediate reasoning outputs, but produces a complete, structured execution pipeline that is directly usable for black-box GUI automation. The tight integration between perception, reasoning, and execution ensures that the system maintains

both interpretability and practical applicability, bridging the gap between high-level intent understanding and low-level hardware interaction.

6.1.4 Robustness Under Edge Condition

The robustness evaluation of the proposed agentic automation system focuses on its behavior under non-ideal and uncertain real-world GUI conditions, where perception noise, interface variability, and ambiguous instructions are commonly observed. The analysis considers how the integrated components—visual perception, LLM-based reasoning, Agentic AI decision-making, and HID execution—collaboratively respond to such edge cases within the closed-loop system.

Missing UI Element Conditions:

When required UI components were not detected in the visual perception output, the system demonstrated reliable failure handling through its perception-grounded validation mechanism. The visual perception module correctly reported missing or unrecognized elements, and the Agentic AI controller used this information to prevent unsafe execution. Instead of forwarding invalid actions to the HID layer, the system halted the execution pipeline and generated structured failure logs. These logs included missing element details, reasoning justification, and execution infeasibility status. This behavior ensures safe operation in black-box environments where incorrect interactions could lead to unintended system behavior.

Ambiguous Instruction Conditions:

In scenarios involving vague or incomplete user instructions, the LLM and prompt engineering module played a key role in interpreting intent and generating candidate action sequences. The Agentic AI Logic module then evaluated these outputs against real-time perception data before execution. Although the system was able to infer reasonable task flows in most cases, additional reasoning cycles were required to resolve ambiguity, resulting in increased latency. The perception module provided contextual grounding that helped reduce hallucinated actions and improved alignment with actual UI structure.

A key strength of the system is its transparency across all edge conditions. Even when execution fails or is partially incomplete, the system consistently produces structured outputs containing perception state, LLM reasoning traces, Agentic AI decisions, and HID execution results. This explainability improves debugging efficiency, enhances trust in automated decision-making, and supports post-execution analysis in complex testing environments.

6.1.5 Comparative Behavior Across Scenario Types

A comparative evaluation across different task categories reveals distinct behavioral patterns arising from the interaction between perception accuracy, LLM reasoning complexity, agentic coordination, and HID execution reliability.

Deterministic Structured Tasks:

These tasks demonstrated the highest efficiency and stability. When UI elements were clearly detected by the perception module and instructions were explicit, the LLM generated minimal but accurate action plans. The Agentic AI module quickly validated these actions, and the HID execution layer completed tasks with minimal retries. This tight coupling between perception and instruction resulted in low latency and high execution reliability.

Ambiguous Language Tasks:

Tasks with vague or high-level instructions required deeper semantic interpretation from the LLM module. The Agentic AI Logic system relied heavily on prompt engineering strategies and contextual perception data to infer correct actions. While most tasks were successfully completed, additional reasoning cycles increased execution time. This demonstrates the system's ability to generalize across flexible input formats, supported by stronger computational overhead in both LLM reasoning and agentic validation stages.

State-Dependent Sequential Tasks:

Tasks involving multi-step UI transitions exposed limitations in long-term context tracking. Although the perception module continuously updated UI state after each action, maintaining consistency across multiple steps remained challenging. The Agentic AI module occasionally had to re-evaluate prior decisions when perception outputs indicated unexpected UI changes. This resulted in partial plans or simplified execution flows in some cases.

Overall, the comparative results show that system performance is highest in well-structured and visually stable environments. Performance gradually decreases as task complexity, state dependency, and ambiguity increase, highlighting the importance of improved long-term context retention and cross-step state modeling within the agentic framework.

6.1.6 Practical Result Interpretation

From a practical deployment perspective, the proposed agentic system demonstrates strong applicability in repetitive, structured, and form-based GUI automation tasks. The integration of visual perception, LLM-based reasoning, Agentic AI orchestration, and HID execution enables reliable transformation of natural language instructions into executable system actions without requiring internal system access.

In well-defined scenarios, the system effectively performs end-to-end automation, starting from screen perception and instruction interpretation to structured action planning and hardware-level execution. The perception module ensures accurate UI detection, the LLM provides flexible reasoning capabilities, the Agentic AI module enforces decision consistency, and the HID layer enables real-world interaction. Together, these components create a complete and functional automation pipeline.

However, limitations become more apparent in complex, dynamic, or loosely defined environments. Tasks requiring long-horizon reasoning or persistent state tracking result in higher latency and reduced execution confidence. These challenges are primarily caused by dependency on real-time perception quality and limitations in contextual memory within the LLM reasoning process.

Despite these constraints, the system remains effective for black-box automation scenarios where traditional API-based tools cannot operate. The results confirm that combining perception-driven grounding with agentic decision-making significantly improves adaptability compared to static automation systems. Future enhancements should focus on improving memory persistence, enhancing multi-step reasoning stability, and optimizing feedback-driven re-planning strategies for more complex workflows.

6.2 Research Finding

This section presents the key research findings derived from the implementation and experimental evaluation of the proposed agentic automation system. The findings are based on observed system behavior across full end-to-end execution cycles, supported by empirical evidence from structured JSON run logs, visual perception outputs, LLM-generated action plans, agentic decision traces, and HID execution feedback. The analysis considers the interaction between all four core components: visual perception, LLM-based reasoning and prompt engineering, Agentic AI logic, and HID-based execution.

Visual-context-grounded perception improves decision accuracy in GUI automation:

The integration of the visual perception module with the agentic decision pipeline significantly improved the accuracy of action generation. By extracting structured UI information such as element labels, types, coordinates, and state indicators, the system was able to ground decisions in real-time screen context. The Agentic AI module utilized this perception output to validate and refine LLM-generated plans, ensuring that actions corresponded to actual interface elements rather than relying on generic language interpretation. This tight coupling between perception and decision-making reduced invalid actions and improved execution correctness.

Agentic orchestration enhances coordination across system components:

The Agentic AI Logic module played a critical role in coordinating perception, LLM reasoning, and HID execution within a unified control loop. Instead of allowing

independent component behavior, the agent enforced structured workflow execution through the Perceive–Plan–Act–Evaluate cycle. This centralized orchestration ensured that each stage was aligned with real-time system state, improving consistency and reducing disjointed execution behavior commonly observed in non-agentic automation systems.

Closed-loop validation improves execution safety and system reliability:

The inclusion of a feedback-driven evaluation mechanism significantly enhanced system safety. After each HID execution, the updated screen state from the perception module was used to validate whether the intended outcome was achieved. When mismatches or missing UI elements were detected, the Agentic AI logic halted execution or triggered re-planning instead of continuing blindly. This validation-first approach reduced incorrect interactions and ensured safer operation in black-box GUI environments where internal system access is unavailable.

LLM-driven planning enables both structured and flexible instruction handling:

The LLM and prompt engineering module successfully supported interpretation of both explicit and ambiguous natural language instructions. In coordination with the Agentic AI controller, it generated structured, machine-readable action sequences that could be reliably executed by the HID layer. While structured instructions resulted in direct and efficient planning, ambiguous inputs required additional reasoning cycles and context enrichment. This demonstrates the system’s ability to balance usability with deterministic execution requirements.

Confidence-aware decision signals support adaptive control:

The system generated confidence indicators during planning and validation stages, which were strongly correlated with execution success. High-confidence outputs typically resulted in correct and complete execution flows, while low-confidence outputs were associated with missing UI elements, unclear instructions, or partial plans. The Agentic AI module effectively used these signals to adjust execution strategies, trigger retries, or request re-planning, improving overall robustness.

Semantic ambiguity directly affects system latency:

Execution time was found to be heavily influenced by instruction clarity. Even when the number of actions remained similar, ambiguous instructions led to additional reasoning cycles within the LLM and repeated evaluation by the Agentic AI module. This resulted in higher latency compared to deterministic tasks. The finding confirms that performance bottlenecks are primarily driven by semantic interpretation complexity rather than execution complexity.

Long-horizon task execution remains a structural challenge:

While the system performed well in short, structured workflows, reduced stability was observed in multi-step tasks involving long UI sequences and state transitions. Maintaining consistent context across perception cycles was challenging, particularly when intermediate screen states changed unexpectedly. This sometimes led to partial execution plans or reduced confidence in later stages, highlighting a limitation in long-term memory and state tracking within the agentic framework.

End-to-end instruction-to-HID transformation is practically feasible:

The system successfully demonstrated full pipeline execution from natural language input to low-level HID command generation. The Agentic AI module translated LLM-generated plans into structured execution commands, which were then executed through the ESP32-based HID interface. This confirms that agentic architectures can enable real-world GUI automation without requiring internal system access, APIs, or DOM-level integration.

Explainability improves debugging and system trust:

The structured logging mechanism significantly enhanced system interpretability. Each execution cycle produced detailed JSON logs containing perception data, decision traces, action plans, execution outcomes, and evaluation results. This transparency enabled effective debugging, performance analysis, and failure tracing, making the system suitable for both academic validation and practical testing environments.

Modular architecture supports scalability and system evolution:

The system's layered design—comprising visual perception, LLM reasoning, prompt engineering, Agentic AI orchestration, and HID execution—allows independent

component upgrades without affecting the entire pipeline. This modularity ensures long-term maintainability and supports future enhancements such as improved vision models, alternative LLM integrations, and more advanced decision-making strategies.

6.3 Discussion

This section interprets the experimental findings of the proposed agentic automation system in relation to the research objectives, system architecture, and practical implications in screen-aware GUI automation. The discussion focuses on how the integrated components—visual perception, LLM-based reasoning and prompt engineering, Agentic AI decision-making, and HID execution—work together within the perceive–plan–act–evaluate loop to enable end-to-end black-box automation.

6.3.1 Interpretation Against Research Objectives

The primary objective of this research was to design and implement an Agentic AI system capable of autonomously executing GUI tasks from natural language instructions in a black-box environment without relying on APIs or internal system access. The experimental results confirm that this objective has been achieved within the defined scope.

The system demonstrated the ability to:

- Interpret natural language instructions using the LLM and prompt engineering module
- Extract structured visual context through the perception module
- Coordinate decision-making using the Agentic AI Logic module
- Generate executable step-by-step action plans
- Convert plans into HID-based commands for real-world execution
- Validate execution outcomes using a closed-loop feedback mechanism

These results confirm that combining perception, reasoning, and execution within a unified agentic framework enables effective GUI automation in real-world conditions.

6.3.2 Effectiveness of the Proposed Agentic Architecture

The effectiveness of the system is primarily driven by the integration of four tightly coupled components: visual perception, LLM reasoning, Agentic AI orchestration, and HID execution. Unlike traditional automation systems that operate in isolated stages, this architecture enables continuous coordination across all components.

The Perceive–Plan–Act–Evaluate loop allows the system to:

- Continuously update UI understanding using visual perception
- Refine decisions using LLM-assisted reasoning
- Validate actions through Agentic AI logic before execution
- Execute commands using HID-based interaction
- Adjust behavior dynamically based on evaluation results

This design significantly improves adaptability in dynamic UI environments. The Agentic AI module ensures that LLM outputs are not executed blindly but are instead validated against real-time screen context, improving overall reliability and consistency.

6.3.3 Implications for Autonomous Human–Computer Interaction

From a human–computer interaction perspective, the proposed system enables intent-driven automation, where users interact using natural language rather than predefined scripts or manual workflows.

This introduces several practical implications:

- Reduced dependency on technical scripting knowledge for automation
- Simplified interaction for QA engineers and end users
- Improved accessibility for non-technical users
- Potential application in assistive and accessibility systems
- Shift from rule-based automation to goal-based intelligent agents

The system effectively acts as an intermediary between human intent and machine-level execution, translating high-level goals into structured, executable UI interactions.

6.3.4 Reliability–Latency Trade-off in Agentic Execution

A key observation from the evaluation is the trade-off between reasoning depth and execution latency. The system achieves higher reliability when the LLM and Agentic AI modules perform deeper contextual reasoning using perception feedback; however, this increases execution time.

Latency is influenced by:

- Instruction complexity and ambiguity
- Number of required interaction steps
- Frequency of perception updates
- Re-planning and retry cycles in failure cases
- Coordination overhead between components

Deterministic tasks execute efficiently, while ambiguous or multi-step workflows require additional reasoning cycles. This confirms that adaptability in agentic systems naturally introduces computational overhead.

6.3.5 Closed-Loop Feedback as a Core Strength and Limitation

The closed-loop Perceive–Plan–Act–Evaluate design is one of the strongest aspects of the system. It enables continuous correction by validating execution outcomes using updated visual perception after each action.

Strengths include:

- Real-time error detection using perception feedback
- Automatic re-planning through Agentic AI logic
- Improved robustness compared to open-loop automation systems

However, limitations include:

- Propagation of perception errors into planning decisions
- Increased latency due to repeated vision cycles
- Dependence on LLM reasoning quality for recovery strategies

Despite these limitations, the feedback loop significantly enhances system reliability in unpredictable GUI environments.

6.3.6 Robustness, Trust, and Explainability

The system provides strong explainability through structured outputs generated at each stage of execution. All components contribute to a unified logging mechanism, including:

- Visual perception outputs (UI elements and states)
- LLM-generated action plans
- Agentic AI decision logs
- HID execution traces
- Evaluation and retry reports

This transparency enables:

- Clear debugging of failed executions
- Traceability of decision-making steps
- Identification of perception or execution errors
- Confidence-based interpretation of results

Such explainability is essential for building trust in autonomous systems, particularly in QA automation and black-box testing environments.

6.3.7 Threats to Validity

The findings of this study are subject to several limitations:

- **Limited scenario coverage:** Evaluation was performed on controlled UI workflows and may not represent full real-world diversity

- **Perception dependency:** System performance is highly dependent on visual accuracy from the perception module
- **LLM variability:** Different model configurations may produce variations in reasoning quality
- **Latency inconsistency:** Execution time varies due to iterative perception and re-planning cycles
- **Environment sensitivity:** HID execution depends on system focus, permissions, and OS-level stability

These factors may influence system performance in broader deployment scenarios.

6.3.8 Practical Deployment Implications

The proposed agentic system is suitable for deployment in structured and repetitive GUI automation tasks such as:

- Form filling and login automation
- Basic navigation workflows
- QA testing in black-box applications
- Legacy systems without API access
- Repetitive interaction-based automation tasks

For effective real-world deployment, the following considerations are important:

- Confidence-based execution control for safer automation
- Stable screen focus management for HID reliability
- Efficient triggering of perception cycles to reduce latency
- Structured logging for debugging and auditability
- Prompt optimization to reduce ambiguity in LLM reasoning

7. CONCLUSIONS

This research set out to design and validate an Agentic AI-based screen-aware automation system capable of executing user interface tasks from natural language instructions using a closed-loop perceive–plan–act–evaluate architecture. The main objective was to determine whether an integrated agentic framework combining visual perception, LLM-based reasoning with prompt engineering, centralized agentic control, and HID-based execution can reliably perform real-world UI automation in black-box environments without requiring system-level access. Based on the implementation and experimental evaluation, this objective has been successfully achieved within the defined scope.

The developed system demonstrated a complete end-to-end autonomous workflow across all integrated components. Natural language instructions were first interpreted by the LLM and prompt engineering module, which generated structured reasoning outputs and initial action plans. These plans were then grounded using real-time visual perception data extracted from the screen, ensuring alignment with actual UI elements and states. The Agentic AI Logic module acted as the central orchestrator, validating and refining these outputs into executable sequences. Finally, the HID execution layer translated these validated actions into real-world input commands, while the evaluation stage used updated perception data to confirm execution success and complete the agentic loop.

A key contribution of this work is the integration of a unified closed-loop agentic architecture that tightly coordinates perception, reasoning, decision-making, and execution. Unlike traditional automation systems that rely on static scripts or isolated modules, the proposed system continuously evaluates execution outcomes and adapts its behavior in real time. The Agentic AI module ensures that LLM-generated outputs are not executed blindly but are validated against live visual context, enabling safe re-planning and recovery when failures or inconsistencies are detected.

Another important outcome is the effective collaboration between visual-context grounding and LLM-based reasoning. The system successfully maps high-level natural

language instructions to real UI elements detected through perception, enabling accurate and executable action generation. Even in cases of ambiguous or incomplete instructions, the LLM module supported the Agentic AI controller in inferring reasonable task sequences, while visual perception ensured that decisions remained grounded in actual interface states. This combination significantly improves reliability compared to standalone reasoning or perception systems.

From a usability perspective, the system demonstrates that natural language can serve as a practical control interface for UI automation. Users can define tasks in intuitive language, while the underlying agentic system handles decomposition, planning, execution, and validation. This reduces dependency on manual scripting, test automation frameworks, or API-level integration, making the system suitable for QA automation, accessibility tools, and legacy system interaction.

However, experimental results also revealed a trade-off between adaptive reasoning and execution efficiency. While the system performs reliably in structured and well-defined tasks, ambiguous or multi-step workflows introduce additional reasoning cycles across the LLM, Agentic AI, and perception components. These cycles increase overall latency, highlighting the inherent balance between intelligence and real-time performance in agentic systems.

From a research perspective, this study validates the core hypothesis that an integrated agentic architecture combining visual perception, LLM-driven reasoning, centralized decision-making, and HID-based execution can support practical UI automation in black-box environments. The modular structure of the system—comprising perception, reasoning, agentic control, and execution layers—ensures scalability and allows future enhancements without redesigning the entire pipeline.

Despite its effectiveness, several improvement areas remain. These include enhancing long-horizon task memory for better workflow continuity, optimizing prompt and reasoning efficiency to reduce latency, and improving perception robustness under noisy

or dynamic screen conditions. Strengthening state persistence across multi-step interactions will also be critical for more complex real-world deployments.

In conclusion, this research presents a fully functional and experimentally validated Agentic AI framework for screen-aware automation. The system demonstrates that combining natural language reasoning, structured visual grounding, centralized agentic control, and hardware-level execution can produce reliable, interpretable, and adaptive automation behavior. This work provides a strong foundation for future research in autonomous UI agents, intelligent automation systems, and next-generation human–computer interaction frameworks.

8. REFERENCES

- [1] M. Harman, P. McMinn, J. T. de Souza, and S. Yoo, "Search based software engineering: Techniques, taxonomy, tutorial," Empirical Software Engineering, 2012.
- [2] A. Mesbah, A. van Deursen, and S. Lenseslink, "Crawling Ajax-based web applications through dynamic analysis of user interface state changes," ACM Transactions on the Web, 2012.
- [3] R. Smith, "An overview of the Tesseract OCR engine," Proc. ICDAR, 2007.
- [4] T. Brown et al., "Language Models are Few-Shot Learners," Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [5] S. Newman, "Building Microservices: Designing Fine-Grained Systems," O'Reilly Media, 2015.
- [6] D. Amershi et al., "Software Engineering for Machine Learning: A Case Study," ICSE, 2019.
- [7] S. M. M. K. Rahman and A. M. Memon, "The forgotten GUI testing: A systematic mapping study," in 2018 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), 2018, pp. 1-10.
- [8] W3C, "WebDriver," W3C Recommendation, Jun. 2018. [Online]. Available: <https://www.w3.org/TR/webdriver/>
- [9] S. R. Choudhary, H. Versee, and A. Orso, "WEBDIFF: Automated identification of cross-browser issues in web applications," in 2010 IEEE International Conference on Software Maintenance, 2010, pp. 1-10.
- [10] R. Smith, "An overview of the Tesseract OCR engine," in Proc. 9th International Conference on Document Analysis and Recognition (ICDAR), 2007, pp. 629-633.
- [11] B. Deka, Z. Huang, and R. Kumar, "Rico: A mobile app dataset for building data-driven design applications," in Proc. 30th Annual ACM Symposium on User Interface Software and Technology (UIST), 2017, pp. 845-854.
- [12] T. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 1877-1901.

- [13] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24824-24837.
- [14] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [15] S. Zhou et al., “WebArena: A realistic web environment for building autonomous agents,” *arXiv:2307.13854*, 2023.
- [16] USB Implementers Forum, “Device Class Definition for Human Interface Devices (HID),” Version 1.11, 2001.
- [17] Espressif Systems, *ESP32 Technical Reference Manual*, Espressif Systems, 2023.
- [18] Yunpeng Song, Y. B. Y. T. G. M. Z. C. (n.d.). *VisionTasker: Mobile Task Automation Using Vision Based UI Understanding and LLM Task Planning*.
- [19] Panneer Selvam Viswanathan. (n.d.). *Prompt Engineering for Conversational AI Systems: A Systematic Review of Techniques and Applications*.
- [20] gaurav gupta. (n.d.). *Vision-Language-Action (VLA) Models: LLMs for robots*.
- [21] Rupesh Garg. (2025). *How to Choose the Best LLM Tools for Your Test Automation Strategy*.
- [22] Wang, L., Chen, X., Deng, X., Wen, H., You, M., Liu, W., Li, Q., & Li, J. (2024). Prompt engineering in consistency and reliability with the evidence-based guideline for LLMs. *Npj Digital Medicine*, 7(1), 41. <https://doi.org/10.1038/s41746-024-01029-4>
- [23] S, J. P. B., D, B. K., A, P. S., & D, M. G. (2025). *Prompt Engineering for Large Language Models: A Systematic Review and Future Directions*. <https://doi.org/10.21203/rs.3.rs-6551106/v1>
- [24] Marvin, G., Hellen, N., Jjingo, D., & Nakatumba-Nabende, J. (2024). *Prompt Engineering in Large Language Models* (pp. 387–402). https://doi.org/10.1007/978-981-99-7962-2_30
- [25] Pawlik, L. (2025). *How the Choice of LLM and Prompt Engineering Affects Chatbot Effectiveness*. *Electronics*, 14(5), 888. <https://doi.org/10.3390/electronics14050888>

9. APPENDICES

Appendix - A: Plagiarism report



Class Profile

Now Viewing: HOME > 4TH YEAR IT 2ND SEMESTER 2020 > 4TH YEAR IT 2ND SEM ASSIGNMENT



Submit work
4th Year IT 2nd sem Assignment ▾

 Help

 You have reached your limit of 3 submissions. You can submit again 24 hours after your first submission.

Your work Resubmit

Title	Submitted	Grade	Similarity	Feedback	More
group thesis	Apr 26, 2020 13:54 PM	0 / 100	5%		